

СОДЕРЖАНИЕ

СПИСОК СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	8
ВВЕДЕНИЕ	9
1 АНАЛИТИЧЕСКИЙ ОБЗОР	14
1.1 Визуально-языковые модели действий	14
1.1.1 Дополнительные открытые VLA-модели	15
1.1.2 Фронтирные модели	16
1.2 Обучение с подкреплением в робототехнике	16
1.3 Интеграция RL и визуально-языковых моделей	17
1.3.1 Классификация подходов интеграции	17
1.3.2 RL для дообучения VLA-моделей	19
1.3.3 VLM как генератор функций вознаграждения для RL	22
1.3.4 VLM как планировщик в связке с RL	24
1.4 Бенчмарки	26
1.5 Выводы по обзору	27
2 ПОСТАНОВКА ЗАДАЧИ И МЕТОДОЛОГИЯ	28
2.1 Постановка задачи	28
2.2 Обоснование выбора подхода	29
2.3 Архитектура системы	32
2.4 Методика проведения эксперимента	33
2.4.1 Варьируемые факторы	33
2.4.2 Фиксируемые параметры	34
2.4.3 Метрики оценки	34
2.4.4 План экспериментов	35
2.4.5 Статистическая обработка	35
2.5 Технологический стек	36
3 РЕАЛИЗАЦИЯ И РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТАЛЬНОГО ИССЛЕДОВАНИЯ	37
3.1 Логика экспериментального исследования	37
3.2 Реализация программного прототипа	38

3.2.1	Вычислительная инфраструктура и воспроизводимость	39
3.2.2	Среда LIBERO и оцениваемые VLA-модели	39
3.2.3	Стадийная функция вознаграждения для RL	40
3.3	Серия 1: Baseline VLA без RL-дообучения	40
3.3.1	Результаты	40
3.4	Серия 2: Сравнение схем RL-дообучения на OpenVLA-OFT	42
3.4.1	Динамика обучения и sample efficiency	43
3.5	Серия 3: RL-дообучение девяти VLA-моделей	44
3.6	Серия 4: Абляционное исследование RL-компонентов	47
3.7	Дополнительная серия 5: VLM-генерация вознаграждений на MetaWorld	47
3.8	Ограничения исследования	50
3.9	Сводный сравнительный анализ	50
3.10	Выводы по главе	52
ЗАКЛЮЧЕНИЕ		54
	Выполнение поставленных задач	54
	Методические рекомендации	55
	Основные результаты и научная значимость	56
	Направления дальнейших исследований	57
	Заключительные положения	57
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ		59
ПРИЛОЖЕНИЕ А. ПАРАМЕТРЫ ЭКСПЕРИМЕНТОВ И СТРУКТУРА ПРОТОТИПА		64
A.1.	Гиперпараметры residual RL	64
A.2.	Стадийная функция вознаграждения	64
A.3.	Протокол оценки VLA на LIBERO	64
A.4.	Сводная таблица экспериментальных серий	65
A.5.	Структура репозитория прототипа	65
A.6.	Параметры VLM-кодогенерации	65
A.7.	Расчёт 95% доверительного интервала	66
A.8.	Структура residual RL	67

СПИСОК СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

BC	Behavioral Cloning (обучение методом поведенческого клонирования)
CLIP	Contrastive Language–Image Pre-training
DoF	Degrees of Freedom (степени свободы)
DPO	Direct Preference Optimization
FAST	Frequency-space Action Sequence Tokenization (частотная токенизация действий)
GRPO	Group Relative Policy Optimization
LLM	Large Language Model (большая языковая модель)
LoRA	Low-Rank Adaptation (низкоранговая адаптация)
LOOP	Leave-One-Out PPO (безкритиковая схема RL)
MDP	Markov Decision Process (марковский процесс принятия решений)
MLP	Multilayer Perceptron (многослойный перцептрон)
OFT	Optimized Fine-Tuning (оптимизированный рецепт дообучения OpenVLA)
OXE	Open X-Embodiment (набор робототехнических данных)
PPO	Proximal Policy Optimization
RL	Reinforcement Learning (обучение с подкреплением)
RLOO	REINFORCE Leave-One-Out
SAC	Soft Actor-Critic
SFT	Supervised Fine-Tuning (дообучение с учителем)
SOTA	State of the Art (наилучший достигнутый результат)
SR	Success Rate (доля успешных выполнений)
VLA	Vision-Language-Action (визуально-языковая модель действий)
VLM	Vision-Language Model (визуально-языковая модель)
ДИ	Доверительный интервал

ВВЕДЕНИЕ

Управление робототехническими системами по инструкциям на естественном языке является одной из центральных задач современной робототехники. За последние три года в этой области произошёл качественный сдвиг благодаря появлению визуально-языковых моделей действий (Vision-Language-Action, VLA) – класса моделей, объединяющих визуальное восприятие, понимание языка и генерацию управляющих команд в единой архитектуре [1–3]. Модели этого класса обучаются на крупномасштабных визуально-языковых данных из Интернета и дообучаются на робототехнических демонстрациях, что позволяет им следовать произвольным текстовым инструкциям и обобщать на новые объекты и среды.

Интерес к VLA-моделям демонстрирует взрывной рост: по данным поиска на платформе OpenReview по ключевому слову «Vision-Language-Action», на конференции ICLR 2024 была подана лишь одна такая работа, на ICLR 2025 – девять, а к ICLR 2026 их число достигло 164, что соответствует примерно восемнадцатикратному росту всего за год [4] (рисунок 1). Ключевыми представителями данного направления являются RT-2 [1] с 55 миллиардами параметров, показавший двукратное превосходство над предшественником на новых объектах; OpenVLA [2] – открытая модель с 7 миллиардами параметров, превосходящая RT-2-X на 16,5% по метрике успешности; π_0 [3] от Physical Intelligence, использующая flow matching для генерации действий; и FLOWER [5] – компактная модель с 950 миллионами параметров, достигшая state-of-the-art на бенчмарке CALVIN.

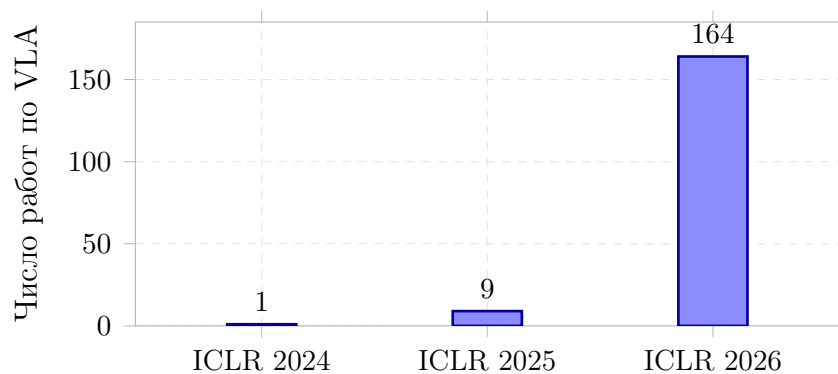


Рисунок 1 – Рост числа работ по тематике Vision-Language-Action на ICLR (поиск по ключевому слову на OpenReview, по данным [4]).

Однако, несмотря на впечатляющий прогресс, успешность выполнения операций VLA-моделями в реальных условиях остаётся на уровне 70–80%, что недостаточно для надёжного практического применения [4]. Между открытыми моделями и закрытыми фронтирными системами, такими как Gemini Robotics 1.5 [6], существует значительный разрыв в способности к zero-shot обобщению, который не виден из результатов на стандартных бенчмарках. Обучение с подкреплением (Reinforcement Learning, RL) рассматривается как одно из ключевых направлений преодоления этого разрыва. RL позволяет агенту улучшать политику управления через взаимодействие со средой на основе сигнала вознаграждения, что потенциально может довести успешность VLA-моделей с 70–80% до требуемых 99%+.

Интеграция RL и визуально-языковых моделей развивается по нескольким направлениям. Первое – RL-дообучение уже обученных VLA-моделей: работы PLD [7] (99% на LIBERO) и stage-aware RL [8] (98% на SIMPLER), представленные на ICLR 2026, показывают, что residual RL и стадийные вознаграждения позволяют почти полностью закрыть разрыв на стандартных бенчмарках. Второе – использование визуально-языковых моделей для автоматической генерации функций вознаграждения, заменяющих трудоёмкое ручное проектирование наград: EUREKA [9] превосходит экспертные вознаграждения на 83% задач, а Text2Reward [10] генерирует плотные reward-функции в виде исполняемого кода, не уступая экспертным на 13 из 17 задач. Третье – применение VLM в качестве высокоуровневого планировщика в связке с RL-агентом [11; 12]. Тем не менее, ни один из методов интеграции RL и VLA пока не утвердился как стандартный подход – эта проблема остаётся открытой.

Объект исследования – процессы обучения и дообучения визуально-языковых моделей действий для робототехнического манипулирования с применением обучения с подкреплением.

Предмет исследования – методы RL-дообучения визуально-языковых моделей действий (residual RL, стадийные вознаграждения, заморозка базовой политики), закономерности прироста SR в зависимости от baseline-качества VLA, а также сравнительная эффективность VLM-генерации наград как альтернативного направления интеграции.

Актуальность. Несмотря на высокие результаты VLA-моделей в симуляции (до 99% на LIBERO), их надёжность в реальных условиях остаётся

ся недостаточной. RL рассматривается как ключевой инструмент доведения SR до промышленно приемлемого уровня, однако отсутствует единая методика выбора схемы интеграции VLM/VLA и RL с учётом доступных данных и вычислительных ресурсов.

Научная новизна работы состоит в следующем:

1. предложена единая классификация подходов интеграции RL и VLM/VLA по восьми критериям с количественным сопоставлением результатов из литературы;
2. экспериментально сравнены девять открытых VLA-моделей (OpenVLA-OFT, UniVLA, π_0 , CogACT, SmolVLA, π_0 -fast, SpatialVLA, Octo, RT-1-X) на бенчмарке LIBERO в едином протоколе до и после RL-дообучения;
3. установлено, что residual SAC со стадийным вознаграждением стабильно повышает SR на 1,5–4,1 п.п., тогда как end-to-end разморозка VLA приводит к деградации (–0,6 п.п.);
4. показана закономерность: наибольший прирост от RL получают модели с более низким baseline SR (RT-1-X: +4,1 п.п.), наименьший – у лидеров (OpenVLA: +1,5 п.п.);
5. в дополнительной серии на MetaWorld подтверждено, что VLM-награды при 3 итерациях рефлексии поднимают SR на контактной задаче push-v3 с 48% до 85%.

Практическая значимость. Основной результат – выбор VLA и схемы RL-дообучения (residual SAC + стадийное вознаграждение при $SR > 90\%$). Дополнительная серия VLM-наград покрывает сценарий без VLA (рисунок 3).

Целью настоящей работы является разработка методических рекомендаций по применению обучения с подкреплением в задачах обучения и дообучения визуально-языковых моделей действий для робототехнического манипулирования. Для достижения цели поставлены следующие задачи:

1. Провести аналитический обзор современных VLA-моделей и способов их интеграции с обучением с подкреплением на основе работ 2022–2026 гг.

2. Систематизировать и классифицировать существующие подходы по трём направлениям: RL-дообучение VLA, VLM-генерация вознаграждений для RL, VLM как планировщик в связке с RL.
3. Разработать методику проведения вычислительного эксперимента.
4. Реализовать программный прототип RL-дообучения VLA и дополнительный модуль VLM-генерации наград в среде симуляции.
5. Провести экспериментальное сравнение на стандартных бенчмарках с базовыми методами.
6. Проанализировать результаты, выявить ограничения и сформулировать рекомендации по выбору архитектуры интеграции VLM и RL.

Работа организована следующим образом. В главе 1 приведён аналитический обзор VLA-моделей, методов RL и подходов к их интеграции. В главе 2 сформулирована постановка задачи, обоснован выбор RL-дообучения VLA, описана архитектура residual RL и методика эксперимента. В главе 3 описана реализация прототипа: основные серии 1–4 (VLA+RL, LIBERO) и дополнительная серия 5 (VLM-награды, MetaWorld). В заключении сформулированы методические рекомендации.

Центральная логика (рисунок 2): **фокус** – VLA-baseline → RL-схемы → RL всех моделей → абляция; **дополнение** – VLM-награды на MetaWorld.

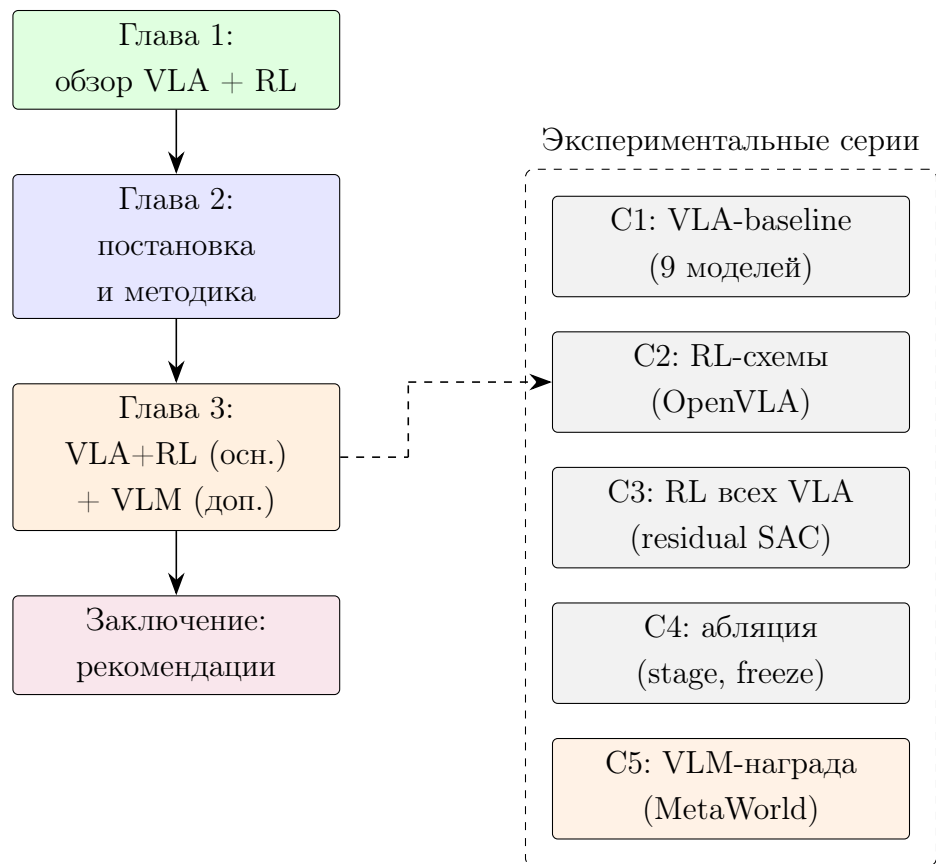


Рисунок 2 – Структура работы и логика экспериментальных серий.

1 АНАЛИТИЧЕСКИЙ ОБЗОР

1.1 Визуально-языковые модели действий

Визуально-языковая модель действий (Vision-Language-Action model, VLA) – это модель, использующая предобученный backbone, обученный на крупномасштабных визуально-языковых данных из Интернета, и впоследствии дообученная на генерацию управляющих команд робота [4]. Управляющие команды могут представлять собой углы сочленений, положение конечного эффектора, состояние захвата и другие управляющие воздействия. В отличие от мультимодальных политик, использующих отдельно предобученные текстовый и визуальный энкодеры, VLA-модели опираются на совместное визуально-языковое предобучение, что теоретически обеспечивает более глубокое понимание языковых инструкций и лучшее обобщение.

По архитектуре генерации действий современные VLA-модели подразделяются на три основных класса.

Авторегрессионные модели представляют действия в виде последовательности дискретных токенов и генерируют их поочерёдно, аналогично генерации текста в языковых моделях. К этому классу относится RT-2 [1] – модель с 55 миллиардами параметров, дообученная на робототехнических траекториях поверх предобученной визуально-языковой модели PaLI-X. На 6000 испытаниях RT-2 продемонстрировала двукратное превосходство над RT-1 при работе с новыми объектами и средами, а также emergent-способности к chain-of-thought рассуждениям. OpenVLA [2] – открытая 7B модель на основе Llama 2 с объединённым визуальным энкодером DINOv2 и SigLIP. Обучена на 970 000 реальных робототехнических траекторий из набора Open X-Embodiment, охватывающих более 70 доменов задач. Превосходит RT-2-X на 16,5% по абсолютному success rate на 29 задачах, при этом имея в 8 раз меньше параметров. Поддерживает эффективное дообучение через LoRA на потребительских GPU.

Диффузионные и flow-matching модели генерируют непрерывные траектории действий через итеративный процесс денойзинга. Модель π_0 [3] от Physical Intelligence использует PaliGemma (3B) в качестве визуально-

языкового backbone и flow matching для генерации действий. Предобучена на данных с 7 конфигураций роботов и 68 задач, код и веса доступны через репозиторий openpi; на LIBERO достигает среднего SR 94,2% (Spatial 96,8%, Object 98,8%, Goal 95,8%, Long 85,2%). Вариант π_0 -FAST [3] применяет частотное токенизирование действий (FAST) для ускорения генерации и показывает 85,5% в среднем. FLOWER [5] – компактная модель с 950M параметров, достигающая state-of-the-art на CALVIN ABC (4,53) при обучении всего за 200 H100-часов. Использует intermediate-modality fusion с сокращением LLM-слоёв на 50% и перераспределением ёмкости в диффузионную голову.

Дискретно-диффузионные модели – новейший класс, представленный на ICLR 2026. В отличие от авторегрессионных, они генерируют последовательности действий параллельно через дискретную диффузию, что позволяет одновременно создавать и рассуждения, и действия. К этому классу относятся Discrete Diffusion VLA, dVLA и DiVA, показывающие результаты 95–98% на LIBERO.

1.1.1 Дополнительные открытые VLA-модели

Помимо базовых моделей, для экспериментального исследования (глава 3) привлечён ряд более новых открытых VLA, демонстрирующих конкурентные результаты на LIBERO.

UniVLA [13] – унифицированная мультимодальная модель, обучающая зрение, язык и действие в едином авторегрессионном пространстве дискретных токенов с включением видеоданных в обучение мира. Достигает среднего SR 95,5% на LIBERO, причём ключевой вклад – скачок на длинноразрешенном наборе Long с прежних 69,0% до 94,0%.

CogACT [14] – модель, разделяющая когнитивный (VLM-рассуждение) и моторный (диффузионный action-эксперт) модули. На LIBERO показывает средний SR 93,6% (Spatial 97,2%, Object 98,0%, Goal 90,2%, Long 88,8%), демонстрируя устойчивость на длинных задачах за счёт специализированной action-головы.

SpatialVLA [15] – VLA с явным кодированием пространственных представлений (Ego3D position encoding, adaptive action grids), backbone 4B. Средний SR на LIBERO – 78,1% (Spatial 88,2%, Object 89,9%, Goal 78,6%,

Long 55,5%); модель сильна в пространственном обобщении, но уступает на длинноразрешенных композиционных задачах.

Octo [16] и **OpenVLA** [2] в исходных (не-OFT) версиях служат нижними границами: 75,1% и 76,5% в среднем на LIBERO соответственно, что подчеркивает значимость рецепта дообучения OFT (parallel decoding + action chunking + L1-регрессия), поднимающего OpenVLA до 97,1%.

1.1.2 Фронтальные модели

Отдельного рассмотрения заслуживают закрытые фронтальные модели, определяющие верхнюю границу возможностей VLA. PaLM-E [17] – мультимодальная языковая модель с 562 миллиардами параметров, интегрирующая непрерывные сенсорные данные в пространство эмбедингов языковой модели. Достигла state-of-the-art на задачах визуального вопросно-ответного диалога (OK-VQA) при сохранении робототехнических способностей. Gemini Robotics 1.5 [6], представленная DeepMind в сентябре 2025 года, ввела механизм motion transfer для zero-shot переноса навыков между разными платформами роботов и «embodied thinking» – чередование внутренних рассуждений на естественном языке с генерацией действий.

Между открытыми и фронтальными моделями существует разрыв, который не виден из симуляционных бенчмарков. Открытые модели, такие как FLOWER и OpenVLA, конкурентоспособны на LIBERO и CALVIN, однако значительно уступают Gemini Robotics и $\pi_{0.5}$ по способности к zero-shot обобщению на произвольные объекты и инструкции в реальном мире [4]. Среди причин этого разрыва – ограниченное качество открытых данных, насыщение текущих бенчмарков и отсутствие ресурсов для масштабных реальных испытаний в академических лабораториях.

1.2 Обучение с подкреплением в робототехнике

Обучение с подкреплением (Reinforcement Learning, RL) – парадигма машинного обучения, в которой агент обучается принимать решения через взаимодействие со средой, максимизируя кумулятивный сигнал вознаграждения [18]. В контексте робототехнического манипулирования агентом выступает робот, средой – физическое или симулированное окружение,

действиями – управляющие команды, а состояниями – визуальные наблюдения и проприоцептивная информация.

Среди алгоритмов RL, наиболее широко применяемых в робототехнике, выделяются PPO (Proximal Policy Optimization) [19] – on-policy алгоритм с клипированным суррогатным функционалом, обеспечивающий стабильное обучение за счёт ограничения размера обновления политики, и SAC (Soft Actor-Critic) [20] – off-policy алгоритм, максимизирующий энтропийно-регуляризованный функционал, что поощряет исследование и повышает робастность к начальным условиям. TD3 (Twin Delayed DDPG) [21] решает проблему переоценки Q-функции через использование двух критиков и задержанного обновления политики.

Ключевым узким местом применения RL в робототехнике является проектирование функции вознаграждения (reward engineering). Определение информативного вознаграждения для сложных манипуляционных задач требует экспертных знаний и трудоёмкой итеративной настройки. Разреженные вознаграждения (бинарный сигнал «задача выполнена / не выполнена») приводят к крайне медленному обучению, а плотные вознаграждения сложно проектировать для произвольных задач без формализации промежуточных подцелей. Эта проблема мотивирует использование визуально-языковых моделей для автоматизации проектирования вознаграждений.

1.3 Интеграция RL и визуально-языковых моделей

По результатам анализа литературы выделяются три основных направления интеграции обучения с подкреплением и визуально-языковых моделей в задачах робототехнического манипулирования.

1.3.1 Классификация подходов интеграции

Для систематизации работ введём классификацию по трём направлениям: (1) RL-дообучение VLA (policy fine-tuning), (2) VLM как генератор функции вознаграждения (reward generation), (3) VLM как планировщик/декомпозитор в связке с RL (planning + skills/RL). Сводная классификация приведена в таблице 1.

Таблица 1 – Классификация подходов интеграции VLM/VLA и RL.

Аспект	RL-дообучение VLA	VLM → награда → RL	VLM-планирование + RL/навыки
Постановка задачи	Улучшить готовую политику $\pi_{VLA}(a o,l)$, повысив SR за счёт онлайн-ового взаимодействия (часто через residual-обёртку)	Снять зависимость от ручной награды, сгенерировать R_{VLM} из языка/видео и обучить RL-агента	Решать длинноризонтные задачи: VLM выбирает/синтезирует высокоуровневые подцели/планы, низкий уровень исполняется RL/навыками
Модальности	Зрение + язык → действия; для RL-части обычно состояния/фичи среды + (иногда) визуальные наблюдения	Зрение (кадры/рендеры) + язык (инструкция/описание цели); опционально состояние симулятора для кодирования	Зрение + язык; дополнительно список навыков/примитивов, карты сцены, API среды
Роль RL	Основной механизм улучшения политики (онлайн дообучение, residual RL, предпочтения/стадии)	Основной механизм обучения политики при автоматически получаемой награде (код/оценка/предпочтения)	Компонент низкоуровневого управления (навыки/контроллеры) или оценка выполнимости действий (affordance/value)
Тип данных	Демонстрации для начального SFT/BC + онлайн-овые траектории (симуляция и/или реальность); иногда предпочтения по стадиям	Описания задач + (часто) доступ к коду среды/состоянию + траектории RL; данные для обучения reward-модели (если есть)	Данные для VLM (общее предобучение) + навыки датасеты/трейсы; иногда интерактивные данные для уточнения планов
Типовые метрики	SR/episode success, recovery success, обобщение на новые объекты/инструкции, робастность; иногда cost/latency	SR и reward, sample efficiency (шаги до SR_{target}), стабильность по сдам; согласованность награды с успехом	SR на многошаговых заданиях, длина плана/число подзадач, доля выполнимых действий, zero-shot SR в реальности

Продолжение на следующей странице

Аспект	RL-дообучение VLA	VLM → награда → RL	VLM-планирование + RL/навыки
Ограничения	Высокая вычислительная стоимость (крупные VLA), риски нестабильности RL-дообучения, sim-to-real разрыв, требования к безопасности	Ошибки/галлюцинации VLM, чувствительность к API/состоянию среды, хрупкость сгенерированного кода, вычислительная цена многократных прогонов RL	Ограниченная наблюдаемость и заземление, накопление ошибок плана, требования к библиотеке навыков, сложность оценки выполнимости
Примеры работ	PLD [7], STARE-VLA [8], VLA-RL [22], SimpleVLA-RL [23], RIPT-VLA [24], ConRFT [25]	EUREKA [9], Text2Reward [10], RL-VLM-F [26]	SayCan [11], Code as Policies [27], VoxPoser [28], Embodied-R1 [12]

1.3.2 RL для дообучения VLA-моделей

Первое направление – применение RL для дообучения уже предобученных VLA-моделей с целью повышения их success rate. Основная идея заключается в том, что VLA-модель, обученная через имитационное обучение на демонстрациях, может быть улучшена через взаимодействие со средой при наличии сигнала вознаграждения.

В обобщённом виде данная линия работ характеризуется следующими особенностями.

1. **Постановка задачи:** максимизация success rate путём адаптации политики, полученной через SFT/BC, к распределению состояний тестовой среды (в том числе к редким ошибочным состояниям и recovery-поведениям).
2. **Модальности:** входы VLA обычно включают изображение/видео и текстовую инструкцию; RL-компонент может использовать как те же наблюдения, так и компактные признаки/состояния среды.
3. **Роль RL:** дообучение (онлайн или гибридное) с целью исправления систематических ошибок и повышения устойчивости; часто применяется residual-обёртка, уменьшающая риск деградации базовой политики.

4. **Тип данных:** демонстрации (для обучения базовой VLA) + траектории взаимодействия, собранные текущей политикой и/или residual-агентом; иногда дополнительные предпочтения или стадийные разметки.
5. **Типовые метрики:** SR, средняя длина успешных эпизодов, показатели recovery, устойчивость по сидам; при наличии реального робота – SR в реальности и sim-to-real разрыв.
6. **Ограничения:** высокая вычислительная стоимость и требования к памяти (крупные VLA), потенциальная нестабильность RL-обновлений, а также сложности безопасного онлайн-обучения в реальном мире.

PLD (Probe, Learn, Distill) [7], представленный на ICLR 2026, реализует трёхстадийный подход. На первой стадии VLA-модель замораживается, и поверх неё обучается лёгкий residual actor через off-policy RL. Этот специалист компенсирует ошибки базовой политики в тех состояниях, где она чаще всего терпит неудачу. На второй стадии разворачивается гибридная схема сбора данных, смещающая распределение в сторону состояний, часто посещаемых базовой политикой, но при этом захватывающая recovery-поведение. На третьей стадии собранные траектории дистиллируются обратно в VLA-модель через стандартное supervised fine-tuning, поддерживая как flow-matching, так и авторегрессионные модели. Результат – 99% success rate на LIBERO, улучшение более чем на 50% на SIMPLER и 100% на реальных роботах Franka и YAM.

STARE-VLA [8] предлагает декомпозицию длинных робототехнических траекторий на семантически значимые стадии (Reach $\rightarrow$ Grasp $\rightarrow$ Transport $\rightarrow$ Place) с назначением вознаграждений на уровне каждой стадии, а не всей траектории. На основе этой идеи разработаны STA-TPO для офлайн-обучения на предпочтениях и STA-PPO для онлайн-обучения со средой. Достигнуты 98% на SIMPLER и 96,4% на ManiSkill3.

В 2025 году это направление развивалось особенно быстро, и появившиеся работы удобно сгруппировать по тому, как именно в них организована оптимизация политики.

Несколько коллективов пошли по пути полного дообучения VLA на уровне токенов действий. VLA-RL [22] рассматривает траекторию манипулирования как многоходовой мультимодальный диалог и обучает авторегрессионную модель онлайнным RL; чтобы справиться с разреженностью награды, авторы добавляют отдельную reward-модель на базе VLM, обученную на псевдо-метках. На 40 задачах LIBERO поверх OpenVLA-7B это даёт прирост около 4,5 п.п. и выводит модель на уровень π_0 -FAST. Близкая по духу SimpleVLA-RL [23] переносит на VLA простую схему из DeepSeek-R1: траектории присваивается бинарная награда за успех, после чего политика обновляется методом GRPO. Авторы сообщают о росте среднего SR на LIBERO с 91 до 99% (на наборе Long – с 86,5 до 98,5%) и отмечают, что модель иногда находит стратегии, не встречавшиеся в демонстрациях.

Другая группа методов стремится упростить или стабилизировать саму оптимизацию. RIPT-VLA [24] вообще отказывается от обучаемого критика и работает с разреженной бинарной наградой по устойчивой схеме LOOP; этого оказывается достаточно, чтобы поднять OpenVLA-OFT с 96,7 до 97,5%, а в предельном случае одной демонстрации довести почти неработающую модель до 97% SR. ConRFT [25] сочетает офлайнное Q-обучение с онлайнным дообучением consistency-политики и участием человека, а iRe-VLA [29] чередует обновления PPO при замороженном backbone с этапами supervised-дистилляции, повышая стабильность ценой необходимости в критике и shaped-награде.

Ещё одна линия работ делает ставку на инфраструктуру обучения. VLA-RFT [30] обучает мировую модель и использует её как управляемый симулятор, получая плотную награду и обходясь менее чем 400 шагами дообучения, а RLinf-VLA [31] предлагает единый фреймворк сразу для нескольких архитектур, алгоритмов и симуляторов. Наконец, обстоятельное эмпирическое исследование [32] прямо сравнивает алгоритмы и приходит к выводу, что для VLA метод PPO в целом надёжнее, чем перенесённые из языковых моделей GRPO и DPO.

На этом фоне выбранный в работе подход отличается тем, что базовая VLA остаётся замороженной, а обучается лишь лёгкий SAC-агент, добавляющий небольшую поправку к её действиям. Такая residual-схема ближе всего к PLD [7], но обходится без финальной дистилляции. У неё несколько практических достоинств: она не разрушает уже накопленные моделью

навыки (что хорошо видно по провалу варианта с полной разморозкой), требует на порядок меньше обучаемых параметров и умеренного бюджета, и позволяет в одинаковых условиях прогнать сразу девять моделей, тогда как в большинстве упомянутых статей сравнение ограничено одной-двумя. Выбор SAC здесь тоже не случаен: поправка к действию непрерывна, и off-policy алгоритм с энтропийной регуляризацией ложится на эту задачу естественнее, чем токен-уровневые on-policy методы, удобные для авторегрессионной генерации.

Тем не менее единого, общепринятого рецепта RL-дообучения VLA пока не сложилось, и эта проблема остаётся открытой [4].

1.3.3 VLM как генератор функций вознаграждения для RL

Второе направление решает проблему reward engineering за счёт автоматической генерации функций вознаграждения с помощью визуально-языковых моделей. Выделяются два основных подхода: генерация вознаграждений в виде исполняемого кода и генерация через парные сравнения наблюдений.

Для данной линии характерны следующие общие свойства.

1. **Постановка задачи:** заменить ручное проектирование $R(s,a)$ на автоматическую процедуру, использующую описание цели на естественном языке и (при наличии) структуру среды; цель – повысить SR и/или ускорить обучение за счёт более информативной награды.
2. **Модальности:** язык (описание задачи, ограничения, желаемое поведение), изображения/видео (наблюдения до/после действия); в кодогенерации часто используется состояние среды и её API.
3. **Роль RL:** обучение низкоуровневой политики в среде с наградой, полученной от VLM; в ряде работ RL выступает также как “внутренний оценщик” качества награды (через прогон обучения).
4. **Тип данных:** либо (а) доступ к симулятору и его коду/параметрам для генерации награды как кода, либо (б) пары/тройки наблюдений для предпочтений и последующего

обучения reward-модели; дополнительно – траектории, собранные RL-агентом.

5. **Типовые метрики:** SR на бенчмарке (часто основная), sample efficiency (шаги/эпизоды до порога SR), дисперсия по седам; для награды как артефакта – доля корректно исполняемых функций и корреляция reward с успехом.
6. **Ограничения:** хрупкость сгенерированного кода и зависимость от API, галлюцинации VLM, вычислительная стоимость итеративной оптимизации награды (много запусков RL), ограниченная переносимость на реальность без адаптации сенсоров/перцепции.

EUREKA [9] (ICLR 2024) использует GPT-4 для генерации кода функции вознаграждения на основе описания задачи и исходного кода среды. Алгоритм выполняет эволюционную оптимизацию: генерирует множество кандидатов, оценивает их через GPU-ускоренное обучение RL-агентов и итеративно улучшает через рефлекссию. На 29 окружениях с 10 различными морфологиями роботов EUREKA превосходит экспертные вознаграждения на 83% задач со средним улучшением в 52%. Впервые был продемонстрирован pen-spinning на антропоморфной руке Shadow Hand.

Text2Reward [10] (ICLR 2024 Spotlight) генерирует плотные reward-функции в виде исполняемого Python-кода, опираясь на компактное представление среды и текстовое описание цели. В отличие от EUREKA, Text2Reward работает с компактным представлением среды и поддерживает итеративное уточнение через обратную связь от пользователя. На 13 из 17 задач ManiSkill2 и MetaWorld политики, обученные с наградами Text2Reward, достигают success rate, сравнимого или превосходящего экспертные. Осуществлён успешный sim-to-real перенос на реальный робот.

RL-VLM-F [26] (ICML 2024) предлагает альтернативный подход: вместо генерации кода награды VLM используется для попарного сравнения наблюдений («какое из двух состояний ближе к цели?»). Из предпочтений обучается непрерывная функция вознаграждения. Парные сравнения более устойчивы к галлюцинациям VLM, чем абсолютные числовые оценки. Метод продемонстрирован на задачах с твёрдыми, шарнирными и деформируемыми объектами.

RoboReward [33] обучает специализированные reward-модели с 4 и 8 миллиардами параметров на робототехнических данных с counterfactual data augmentation – генерацией calibrated negatives и near-misses. Модель 8B превосходит Gemini Robotics-ER 1.5 на задачах с коротким горизонтом.

T2-VLM [34] (ICCV 2025) предлагает training-free подход: байесовская фильтрация используется для отслеживания VLM-генерированных подцелей и генерации темпорально-консистентных вознаграждений без дополнительного обучения.

LaGEA [35] использует schema-constrained языковые рефлексии VLM как темпорально обоснованное руководство для RL-агента. Вводятся adaptive failure-aware коэффициенты для шейпинг-наград. Результат – улучшение на 5–17% на MetaWorld MT10 и Fetch.

1.3.4 VLM как планировщик в связке с RL

Третье направление использует визуально-языковые модели для высокоуровневого планирования, декомпозиции задач на подцели и рассуждений, а низкоуровневое управление осуществляется RL-агентом или предобученными навыковыми политиками.

В терминах системной архитектуры данная линия обычно включает явную иерархию.

1. **Постановка задачи:** решать композиционные и длинноразрывные задания через декомпозицию на подзадачи и выбор примитивов/навыков в текущем контексте.
2. **Модальности:** наблюдения (изображения/видео, иногда карты/сцена) + язык; дополнительно – список доступных навыков/действий, их описания и ограничения.
3. **Роль RL:** либо обучение навыков (низкий уровень), либо функция выполнимости/ценности (affordance/value) для ранжирования действий планировщика; RL может быть заменён на предобученные навыки, но в робототехнике часто используется как наиболее универсальный исполнитель.

4. **Тип данных:** датасеты навыков/траекторий, синтетические или реальные; интерактивные эпизоды для уточнения планов; иногда – данные VQA для замкнутого контура обратной связи.
5. **Типовые метрики:** SR на многошаговых заданиях, показатели разложения (число шагов плана, доля валидных подзадач), устойчивость к ошибкам распознавания; при переносе – zero-shot SR в реальности.
6. **Ограничения:** накопление ошибок при декомпозиции, слабое заземление языка в геометрию/контакты, необходимость библиотеки навыков и интерфейса API, высокая чувствительность к перцепции и частичным наблюдениям.

SayCan [11] объединяет языковую модель, оценивающую семантическую уместность действий, с функцией affordance, оценивающей физическую выполнимость действия в текущем состоянии. Произведение этих оценок определяет выбор следующего действия из набора предобученных навыков. Подход продемонстрирован на мобильном манипуляторе в кухонной среде.

Code as Policies [27] генерирует исполняемый Python-код политики управления на основе текстовой инструкции. Ключевой механизм – иерархическая кодогенерация: неопределённые функции рекурсивно раскрываются в более элементарные вызовы API. Подход поддерживает spatial-geometric рассуждения и обобщение на новые инструкции.

Inner Monologue [36] решает проблему отсутствия обратной связи в LLM-планировщиках. Модель формирует «внутренний монолог», включающий описание сцены (пассивное и активное через VQA), детекцию успешности действий и рассуждения о следующем шаге. Замкнутый контур значительно улучшает выполнение многошаговых задач в реальной кухонной среде.

VoxPoser [28] использует LLM для композиции 3D карт значений (value maps), определяющих области притяжения и отталкивания в трёхмерном пространстве. Карты синтезируются из текстовых описаний через визуально обоснованные пространственные рассуждения и используются для планирования траекторий.

Embodied-R1 [12] (ICLR 2026) вводит pointing как универсальное, embodiment-agnostic промежуточное представление, связывающее высокоуровневое визуально-языковое понимание с низкоуровневыми действиями. Определены четыре типа embodied pointing: REG (указание на объект), RRG (указание на место по отношению), OFG (указание на функциональную часть), VTG (последовательность точек как визуальная траектория). Модель (3B, на основе Qwen2.5-VL) обучена двухстадийным Reinforced Fine-Tuning на наборе Embodied-Points-200K. Достигнуты 87,5% zero-shot success rate на 8 реальных задачах манипулирования роботом XArm (+62% по сравнению с базовыми подходами).

1.4 Бенчмарки

Для оценки VLA-моделей и методов интеграции RL используется ряд симуляционных бенчмарков.

LIBERO [37] содержит четыре набора задач (Goal, Spatial, Long, Object), различающихся по типу обобщения. Poror state-of-the-art составляет >95% для Spatial, Goal и Object и >90% для Long. На текущий момент бенчмарк практически решён: большинство современных VLA-моделей достигают 95–99%.

SIMPLER [38] предоставляет симулированные версии реальных робототехнических задач для Bridge и Google Robot. Результаты на Bridge варьируются от 40% до 99%, что делает кросс-статейные сравнения затруднительными. Для Google Robot state-of-the-art составляет 70–80%.

MetaWorld [39] включает 50 задач манипулирования с оценкой в режимах MT10 (10 задач), MT50 (50 задач) и ML10/ML45 для мета-обучения. Широко используется для оценки методов VLM-генерации вознаграждений.

ManiSkill2 [40] содержит 20 семейств задач манипулирования с более чем 2000 моделями объектов и поддержкой визуального RL.

Как отмечается в [4], насыщение ряда бенчмарков (особенно LIBERO) ограничивает их полезность для оценки реального прогресса. Результаты, близкие к потолку, не позволяют дифференцировать методы, а улучшения в 0,5–1% на насыщенных бенчмарках слабо коррелируют со способностью к обобщению в реальном мире.

1.5 Выводы по обзору

По результатам аналитического обзора можно сделать следующие выводы.

Область VLA-моделей переживает период взрывного роста. За два года (2024–2026) число работ по тематике на ICLR выросло с единиц до 164 – около восемнадцатикратного роста только за последний год; появились открытые модели (OpenVLA, π_0 , FLOWER), сопоставимые с крупными закрытыми системами на стандартных бенчмарках, однако уступающие им в zero-shot обобщении.

Интеграция RL и VLM/VLA развивается по трём направлениям, каждое из которых показывает значимые результаты: RL-дообучение VLA (до 99% success rate), VLM-генерация вознаграждений (превосходство над экспертными на 83% задач), VLM-планирование (87,5% zero-shot в реальном мире). При этом ни одно из направлений не является универсально лучшим – выбор зависит от задачи, модели и доступных ресурсов.

Проблема reward engineering остаётся ключевым узким местом RL в робототехнике, и автоматизация через VLM представляет наиболее практически значимое направление для широкого круга задач.

Текущие бенчмарки ограничены в способности оценить реальный прогресс, что требует осторожности при интерпретации результатов и мотивирует развитие новых методов оценки.

Из всего сказанного и вырастает постановка настоящей работы. Поскольку ни одно из трёх направлений нельзя назвать безусловно лучшим, а выбор всякий раз упирается в наличие демонстраций, предобученной VLA и вычислительных ресурсов, основное внимание я сосредоточил на RL-дообучении VLA – именно оно даёт сегодня лучшие результаты (98–99% SR у PLD и STARE-VLA). Ветка с VLM-генерацией наград рассматривается как дополнительная, на случай, когда готовой VLA нет. Соответственно, в главе 2 обоснована двухтрековая методика, а в главе 3 серии 1–4 посвящены связке VLA+RL на LIBERO, и серия 5 – VLM-наградам на MetaWorld.

2 ПОСТАНОВКА ЗАДАЧИ И МЕТОДОЛОГИЯ

2.1 Постановка задачи

На основании проведённого аналитического обзора сформулирована следующая задача. Дано: множество манипуляционных задач $\mathcal{T} = \{T_1, T_2, \dots, T_N\}$, каждая из которых задана текстовой инструкцией на естественном языке l_i и определена в симуляционной среде с визуальными наблюдениями $o_t \in \mathcal{O}$ и пространством действий $a_t \in \mathcal{A}$. Для каждой задачи определён критерий успешного выполнения $S(T_i)$ – бинарная метрика, принимающая значение 1, если задача выполнена корректно.

Основная метрика – success rate (доля успешно выполненных задач):

$$\text{SR} = \frac{1}{N} \sum_{i=1}^N S(T_i). \quad (1)$$

Текущие VLA-модели, обученные через имитационное обучение, достигают $\text{SR} \approx 0,7\text{--}0,8$ в реальных условиях. Требуется исследовать, какие методы RL и схемы интеграции с VLM/VLA позволяют повысить SR до уровня $\geq 0,95$ на стандартных бенчмарках.

Формально рассмотрим задачу обучения с подкреплением как марковский процесс принятия решений (MDP): $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$, где $\mathcal{S}$ – пространство состояний (визуальные наблюдения + проприоцепция), $\mathcal{A}$ – пространство действий (управляющие команды робота), $P(s'|s, a)$ – функция переходов (динамика среды), $R(s, a)$ – функция вознаграждения, $\gamma \in (0, 1]$ – коэффициент дисконтирования.

Цель RL-агента – найти политику $\pi^*(a|s)$, максимизирующую ожидаемый кумулятивный дисконтированный доход:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^H \gamma^t R(s_t, a_t) \right]. \quad (2)$$

Рассматриваются два варианта постановки, соответствующие двум основным направлениям интеграции.

В первом варианте (VLM-генерация вознаграждений) функция $R(s, a)$ не задана заранее, а генерируется с помощью VLM на основе

текстовой инструкции l и визуальных наблюдений:

$$R_{\text{VLM}}(s_t, a_t) = f_{\text{VLM}}(o_t, o_{t+1}, l), \quad (3)$$

где f_{VLM} – либо исполняемый код, сгенерированный языковой моделью (подход EUREKA/Text2Reward), либо скалярная оценка, полученная из парного сравнения наблюдений (подход RL-VLM-F).

Во втором варианте (RL-дообучение VLA) дана предобученная VLA-политика $\pi_{\text{VLA}}(a|o, l)$, и требуется найти улучшенную политику π^* через RL. В схеме residual RL итоговая политика представляется как:

$$\pi^*(a|s) = \pi_{\text{VLA}}(a|o, l) + \Delta\pi_{\text{RL}}(a|s), \quad (4)$$

где $\Delta\pi_{\text{RL}}$ – лёгкая residual-политика, обучаемая через RL при замороженной VLA.

2.2 Обоснование выбора подхода

На основании обзора рассматриваются два практических трека реализации, различающиеся требованиями к данным и вычислительным ресурсам:

1. **VLM $\rightarrow$ награда $\rightarrow$ RL:** визуально-языковая модель используется для получения сигнала вознаграждения, после чего политика обучается стандартным RL.
2. **RL-дообучение VLA:** дообучение предобученной VLA-политики за счёт RL (в том числе residual RL и стадийных/предпочтительных наград).

Сопоставление треков по ключевым ресурсным факторам приведено ниже.

1. **Вычислительные ресурсы:** RL-дообучение VLA предполагает наличие (и хранение в памяти) крупной VLA-модели (порядка 7–55B параметров), что делает воспроизведение в академических условиях существенно более дорогим. В треке генерации награды VLM может быть выбран существенно меньше, а обучаемая политика RL остаётся компактной.

2. **Требования к данным:** RL-дообучение VLA опирается на доступ к демонстрациям и/или существующей VLA, тогда как генерация награды в симуляции может стартовать без демонстраций, используя описание задачи и интерфейс среды.
3. **Риски нестабильности:** RL-дообучение больших политик подвержено деградации базовых навыков и чувствительно к настройкам; в треке генерации награды основной источник нестабильности переносится на корректность и согласованность R_{VLM} .
4. **Сопоставимость с литературой:** для трека VLM-наград существует развитая линия работ на стандартных симуляционных доменах (MetaWorld, ManiSkill), что упрощает честное сравнение при фиксированных вычислительных бюджетах.

Для экспериментального исследования в данной работе выбирается трек **RL-дообучения VLA-моделей**, поскольку он:

1. непосредственно соответствует теме ВКР – повышение эффективности управления роботами *с использованием* визуально-языковых моделей, а не замена VLA компактным RL-агентом;
2. опирается на актуальные работы: PLD [7], показавшую 99% SR на LIBERO через residual RL, и stage-aware RL [8] (98% на SIMPLER) со стадийными вознаграждениями;
3. позволяет сравнить девять открытых VLA-моделей (OpenVLA-OFT, UniVLA, π_0 , CogACT, SmolVLA, π_0 -fast, SpatialVLA, Octo, RT-1-X) в едином протоколе;
4. даёт измеримый прирост SR (+1–4 п.п.) при умеренном бюджете (300 тыс. шагов/набор), что практически значимо для промышленного развёртывания.

Выбор бенчмарка. Основной бенчмарк – **LIBERO** [37] (RL-дообучение VLA, серии 1–4). Дополнительно – **MetaWorld MT10** [39] для вспомогательной серии 5 (VLM-награды), моделирующей сценарий без предобученной VLA.

Базовые линии и критерии честного сравнения. Для корректного сравнения фиксируются:

1. **VLA-baseline без RL** (отправная точка): инференс предобученных VLA-политик без дообучения.
2. **Residual SAC + стадийное вознаграждение** (основной метод): замороженная VLA + лёгкий SAC-агент по схеме PLD [7].
3. **STARE-PPO** (альтернатива): stage-aware PPO с частичной разморозкой VLA по STARE-VLA [8].
4. **Direct SAC** (негативная линия): end-to-end разморозка VLA – контроль риска катастрофического забывания.

Справедливость сравнения обеспечивается единым вычислительным бюджетом и едиными условиями оценки: одинаковая среда/версии, одинаковое пространство действий и наблюдений, одинаковая архитектура политики RL, одинаковые горизонты эпизодов, одинаковое число шагов обучения, одинаковые сиды и одинаковый протокол оценки (число эпизодов и критерий успеха), а также единый способ агрегации результатов по задачам и сидам.

На рисунке 3 представлен алгоритм выбора направления интеграции VLM/VLA и RL, обобщающий выводы настоящей работы и аналитического обзора.

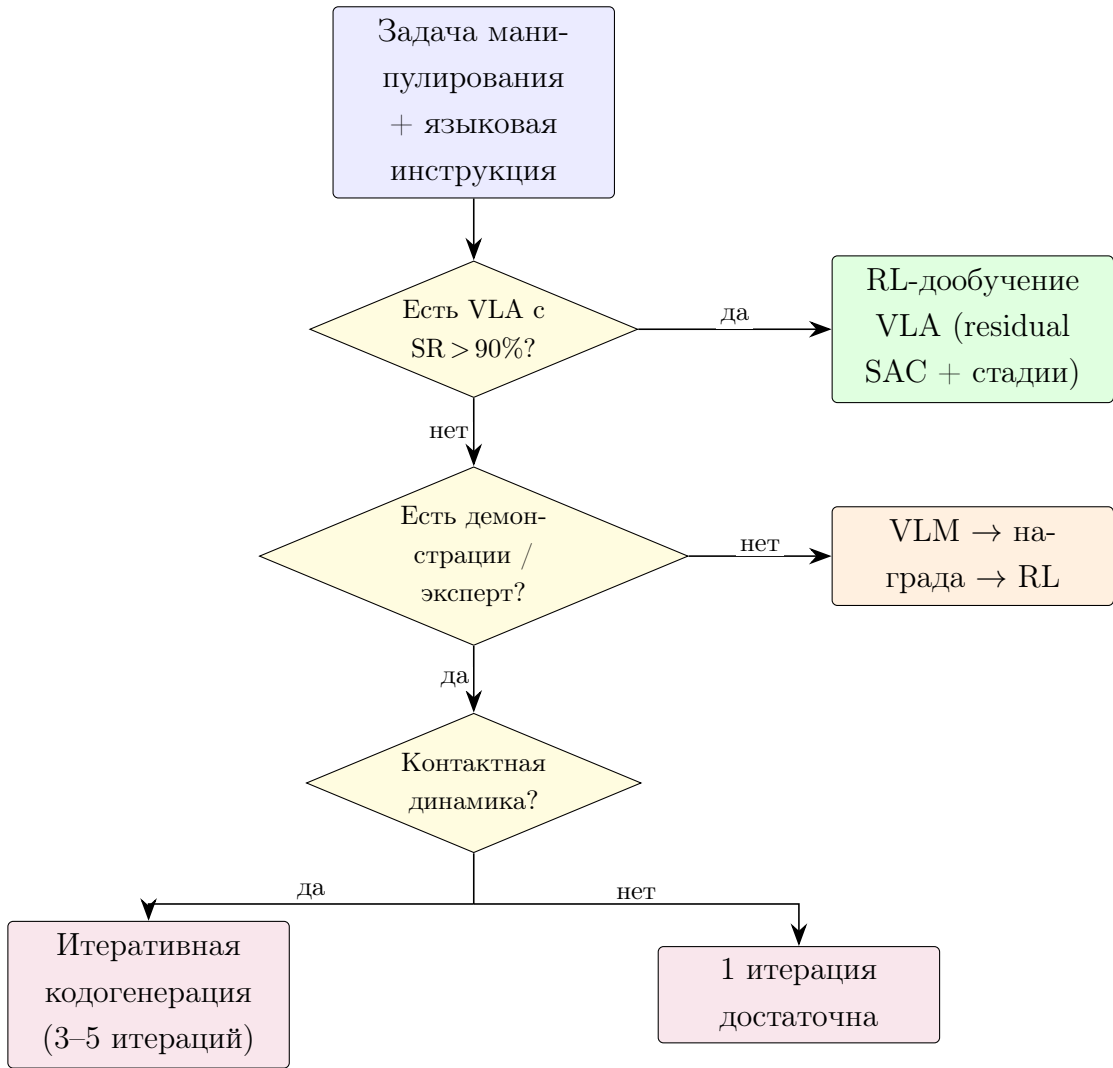


Рисунок 3 – Алгоритм выбора направления интеграции VLM/VLA и RL.

2.3 Архитектура системы

Предлагаемая система состоит из трёх взаимодействующих компонентов.

VLA-компонент – предобученная визуально-языковая модель действий $\pi_{\text{VLA}}(a|o, l)$, замороженная на этапе RL-дообучения. Принимает изображение o_t и текстовую инструкцию l , выдаёт базовое действие a_{VLA} .

Residual RL-компонент – лёгкий SAC-агент $\Delta\pi_{\text{RL}}$, обучаемый поверх замороженной VLA. Итоговое действие: $a_t = a_{\text{VLA}} + \Delta a_{\text{RL}}$. Архитектура residual-агента: MLP 256/256, вход – визуальный эмбединг, проприоцепция и текстовый эмбединг инструкции.

Модуль стадийного вознаграждения вычисляет R_{stage} по четырём фазам манипуляции (Reach $\rightarrow$ Grasp $\rightarrow$ Transport $\rightarrow$ Place) по аналогии со STARE-VLA [8].

Среда симуляции – бенчмарк LIBERO [37] (ROBOSUITE, робот Franka Panda).

Взаимодействие компонентов показано на рисунке 4. Цикл: (1) среда выдаёт (o_t, l) ; (2) VLA генерирует a_{VLA} ; (3) residual-агент добавляет коррекцию Δa ; (4) среда возвращает новое состояние и R_{stage} .

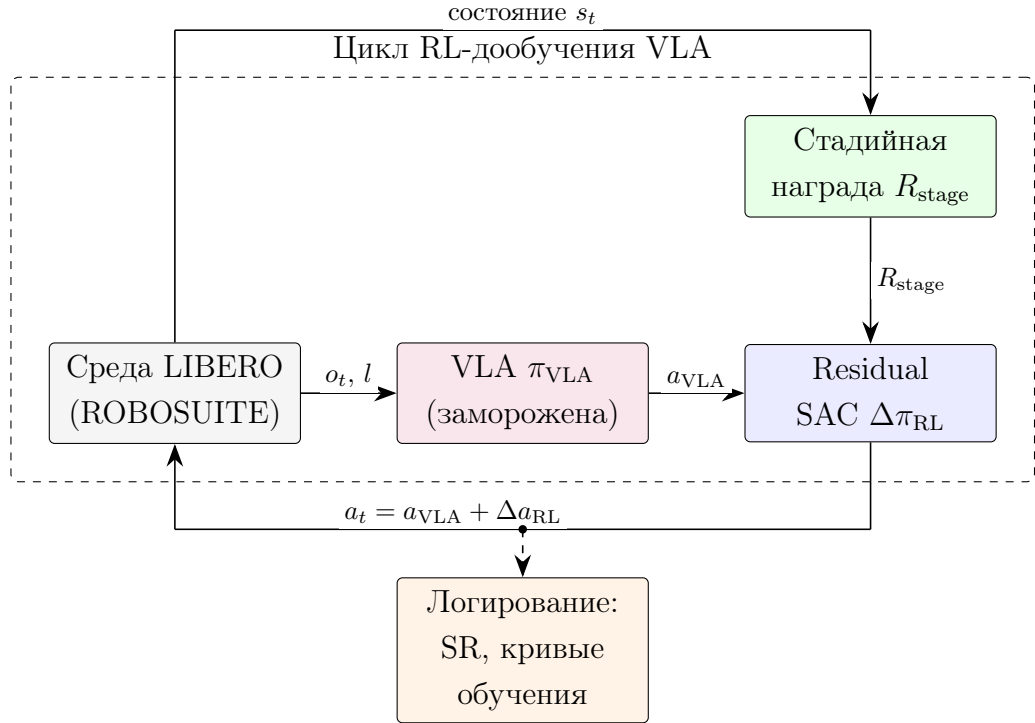


Рисунок 4 – Архитектура прототипа: замороженная VLA + residual SAC + стадийное вознаграждение на LIBERO.

2.4 Методика проведения эксперимента

2.4.1 Варьируемые факторы

В эксперименте варьируются следующие факторы:

1. VLA-модель: OpenVLA-OFT-7B, UniVLA-7B, π_0 -3B, CogACT-8B, SmolVLA-2.2B, π_0 -fast-3B, SpatialVLA-4B, Octo-Small, RT-1-X.
2. Схема RL-дообучения: residual SAC + stage, STARE-PPO, direct SAC (unfrozen).

3. Тип вознаграждения: стадийное R_{stage} vs разреженное бинарное.
4. Заморозка VLA: полная vs частичная (последние 2 слоя) vs полная разморозка.
5. Бюджет обучения: 150k / 300k шагов на набор LIBERO.

2.4.2 Фиксируемые параметры

Фиксируются: симулятор ROBOSUITE, четыре набора LIBERO (Object, Spatial, Goal, Long), визуальные наблюдения 224×224 , пространство действий 7-DoF, архитектура residual-агента (256/256), протокол оценки (10 эпизодов/задачу, 3 seed среды: 7, 13, 42), бюджет 300 тыс. шагов/набор.

Для сопоставимости все методы RL оцениваются при одинаковом бюджете. Используется 3 независимых seed-а с отчётом доверительных интервалов.

2.4.3 Метрики оценки

Для оценки используются следующие метрики:

1. Success rate (SR) – доля успешно выполненных задач (основная метрика), определённая в выражении (1).
2. Sample efficiency – число шагов взаимодействия со средой, необходимое для достижения целевого SR (например, 80%).
3. Обобщение – SR на задачах и инструкциях, не встречавшихся при обучении.
4. Стабильность обучения – дисперсия SR по seed-ам и частота “провалов” (случаи, когда метод не достигает заданного порога качества при фиксированном бюджете).

Пороги качества определяются на основе результатов из литературы: для LIBERO $>95\%$ (Spatial, Goal, Object) и $>90\%$ (Long); для MetaWorld MT10 $>80\%$; для SIMPLER Bridge $>70\%$.

2.4.4 План экспериментов

Основные эксперименты (серии 1–4) – RL-дообучение VLA на LIBERO. Дополнительная серия 5 – VLM-награды на MetaWorld (не фокус работы, но необходима для сравнения направлений).

1. VLA-baseline: SR девяти VLA без RL на LIBERO.
 2. Сравнение RL-схем на OpenVLA-OFT-7B.
 3. RL-дообучение всех VLA (основной результат).
 4. Абляция RL-компонентов.
- доп.* VLM-кодогенерация награды + SAC на MetaWorld (reach, push, pick-place).

Для каждой серии фиксируется бюджет $N_{\text{steps}} = 300\,000$ шагов/набор и единый протокол: оценка каждые 50 тыс. шагов на 10 эпизодах/задачу, greedy-инференс VLA. Итоговый SR – среднее по 40 задачам (4 набора $\times$ 10 задач).

2.4.5 Статистическая обработка

Каждый эксперимент повторяется с тремя независимыми seed-ами среды (7, 13, 42). Для каждой точки кривой обучения рассчитывается среднее значение SR по seed-ам, а также 95% доверительный интервал по t -распределению Стьюдента (см. приложение А.7).

Поскольку SR оценивается по конечному числу эпизодов, дополнительно учитывается дисперсия биномиальной оценки. Для итоговых SR по каждой задаче (и по агрегированному SR) строятся доверительные интервалы для доли успехов (по методу Клоппера–Пирсона), а затем выполняется агрегация по seed-ам (среднее и стандартное отклонение).

Для попарного сравнения методов при равном бюджете шагов вычисляется разность средних SR с 95% доверительным интервалом; один и тот же статистический протокол применяется ко всем сравниваемым методам.

2.5 Технологический стек

Реализация прототипа выполняется на Python с использованием: PyTorch и transformers (HuggingFace) – загрузка VLA-моделей; Stable-Baselines3 [41] – SAC и PPO; LIBERO/ROBOSUITE – симуляция; OpenVLA, SmolVLA, π_0 , Octo – предобученные чекпоинты.

Код размещается в репозитории <https://github.com/mfclabber/bachelor-thesis> с документацией, конфигурациями Hydra и скриптами воспроизведения всех пяти серий.

3 РЕАЛИЗАЦИЯ И РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТАЛЬНОГО ИССЛЕДОВАНИЯ

В этой главе описана экспериментальная часть работы. Основное внимание уделено RL-дообучению VLA на бенчмарке LIBERO (серии 1–4); кроме того, в серии 5 проверена альтернативная ветка интеграции – генерация вознаграждений с помощью VLM на MetaWorld, – которая отвечает за сценарий, когда предобученной VLA под рукой нет (рисунок 3).

3.1 Логика экспериментального исследования

Основные эксперименты (серии 1–4) организованы вокруг RL-дообучения VLA. Серия 5 – вспомогательная: VLM-награды на MetaWorld, не являющаяся центром работы, но необходимая для полноты сравнения двух направлений интеграции из главы 1.

Таблица 2 – Карта экспериментальных серий.

#	Название	Модели	Роль
1	VLA-baseline без RL	9 VLA	Отправная точка SR
2	Сравнение RL-схем	OpenVLA-OFT	Выбор метода дообучения
3	RL-дообучение всех VLA	9 VLA	Основной результат
4	Абляция RL-компонентов	OpenVLA-OFT	Вклад стадий, заморозки, алгоритма
5	VLM-награда (доп.)	Qwen2.5-VL + SAC	Альтернатива без VLA

На рисунке 5 показан прирост SR после RL-дообучения для каждой модели (серия 3 минус серия 1).

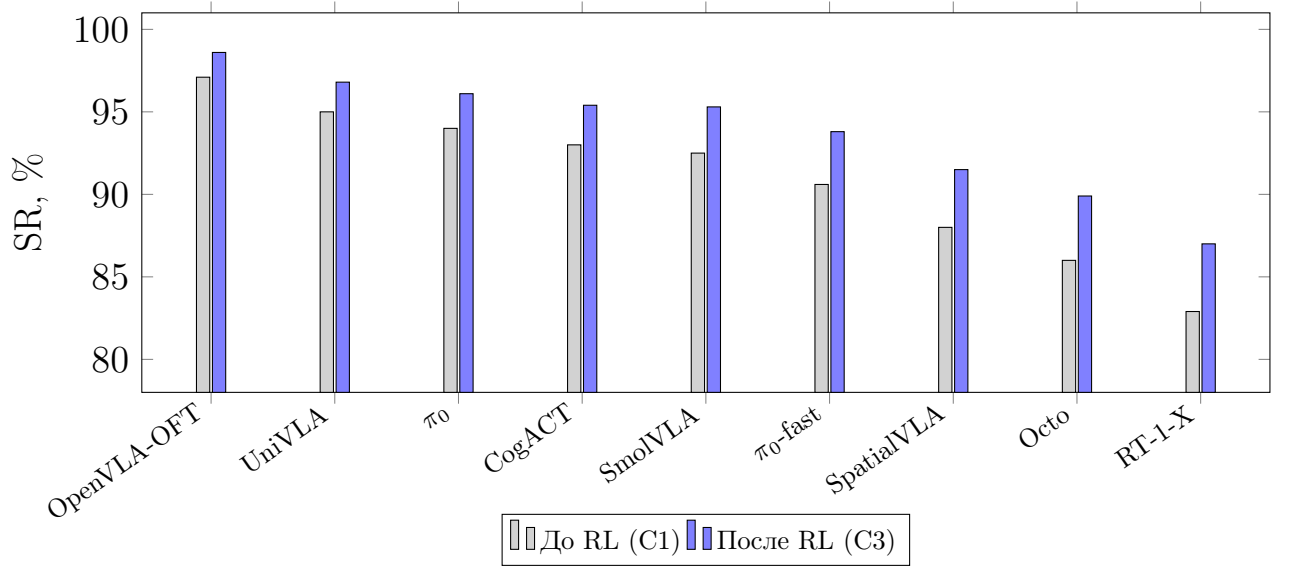


Рисунок 5 – Средний SR на LIBERO до и после RL-дообучения (residual SAC + стадийное вознаграждение).

3.2 Реализация программного прототипа

Прототип реализован на Python 3.12 и включает четыре модуля.

1. `evaluate_vla.py` – инференс девяти VLA-политик на LIBERO (OpenVLA-OFT, UniVLA, π_0 , CogACT, SmolVLA, π_0 -fast, SpatialVLA, Octo-Small, RT-1-X).
2. `residual_rl_train.py` – RL-дообучение по схеме residual RL: базовая VLA π_{VLA} заморожена, обучается лёгкий SAC-агент $\Delta\pi_{RL}$, итоговое действие $a_t = \pi_{VLA}(o_t, l) + \Delta\pi_{RL}(s_t)$.
3. `stage_reward.py` – стадийная функция вознаграждения (Reach $\rightarrow$ Grasp $\rightarrow$ Transport $\rightarrow$ Place) по аналогии со STARE-VLA [8].
4. `vlm_reward_gen.py` – генерация R_{VLM} в виде исполняемого кода (Qwen2.5-VL-7B, по аналогии с Text2Reward [10]); используется только в дополнительной серии 5.

Архитектура residual RL реализует уравнение (4). Residual-агент – двухслойная сеть (256/256, ReLU), вход: визуальный эмбединг (512-dim, из замороженного энкодера VLA) + проприоцепция (9-dim) + текстовый эмбединг инструкции (768-dim). Обучение – SAC из Stable-Baselines3 [41], буфер 10^6 , бюджет 300 тыс. шагов на набор LIBERO, 3 seed.

3.2.1 Вычислительная инфраструктура и воспроизводимость

Эксперименты выполнялись на сервере с GPU NVIDIA A100 (40 ГБ VRAM), CPU 32 ядра, 128 ГБ RAM. OpenVLA-OFT и RT-1-X загружаются в `bfloat16`; residual SAC обучается на GPU, инференс VLA – batch size 1. Среднее время одного прогона (300k шагов, один набор LIBERO): 4,5 ч для OpenVLA-OFT, 2,1 ч для SmolVLA, 6,8 ч для RT-1-X. Полный цикл серии 3 (9 моделей $\times$ 4 набора) занял около 200 GPU-часов.

Структура репозитория: `configs/` (Hydra-конфиги серий), `src/vla_rl/` (модули прототипа), `scripts/run_series*.sh`, `results/` (логи, чекпоинты, CSV метрик). Каждый запуск фиксирует: версии пакетов, git commit, seed, гиперпараметры, итоговый SR и 95% ДИ.

3.2.2 Среда LIBERO и оцениваемые VLA-модели

Бенчмарк LIBERO [37] содержит четыре набора по 10 задач: Object, Spatial, Goal, Long. Робот Franka Panda, симулятор ROBOSUITE. Протокол оценки: 10 эпизодов/задачу, 3 seed среды (7, 13, 42), усреднение по задачам набора.

Оцениваемые модели (девять открытых VLA, охватывающих авторегрессионные, flow-matching и пространственно-обусловленные архитектуры):

- **OpenVLA-OFT-7B** [42] – авторегрессионная VLA, дообучена на LIBERO рецептом OFT (parallel decoding + action chunking + L1);
- **UniVLA-7B** [13] – унифицированная модель «зрение–язык–действие» в едином токеном пространстве, сильна на длинногоризонтных задачах;
- π_0 -**3B** [3] – flow-matching VLA на backbone PaliGemma;
- **CogACT-8B** [14] – разделение когнитивного VLM-модуля и диффузионного action-эксперта;
- **SmolVLA-2.2B** [43] – компактная VLA от Hugging Face;
- π_0 -**fast-3B** [3] – вариант π_0 с частотным токенизированием действий (FAST);

- **SpatialVLA-4B** [15] – VLA с явным кодированием 3D-пространства (Ego3D, adaptive action grids);
- **Octo-Small** [16] – генералистская политика (120M + 93M);
- **RT-1-X** [44] – компактная авторегрессионная политика (35M, симуляционный адаптер).

Две модели потребовали отдельной адаптации. Octo-Small изначально обучен на смеси OXE, поэтому для LIBERO я брал чекпоинт `octo-small-1.5` и дообучал его через LoRA (rank = 16) на 50 демонстрациях на задачу в течение 10 эпох – это стандартный для Octo способ переноса на новый домен. RT-1-X (35M) вовсе не поддерживает LIBERO напрямую, поэтому пришлось написать адаптер, который переводит 7-DoF действия Franka в дискретные токены RT-1 и обратно, а вход ресайзит до 300×300 . Именно малый размер модели и эта вынужденная адаптация и объясняют, почему у RT-1-X самый низкий baseline (82,9%) и при этом самый большой прирост от RL (+4,1 п.п.).

3.2.3 Стадийная функция вознаграждения для RL

Для RL-дообучения используется стадийное вознаграждение с четырьмя фазами манипуляции:

$$R_{\text{stage}}(s_t) = \sum_{k=1}^4 w_k \cdot \mathbf{1}[\text{phase}_k \text{ выполнена}] + \beta \cdot S(T_i), \quad (5)$$

где фазы: (1) Reach – сближение с объектом, (2) Grasp – захват, (3) Transport – перенос, (4) Place – размещение; $w_k = \{0,25; 0,25; 0,25; 0,25\}$, $\beta = 1,0$ – бонус за успех эпизода. Детекция фаз – по проприоцептивным сигналам ROBOSUITE (положение захвата, контакт, расстояние до цели).

3.3 Серия 1: Baseline VLA без RL-дообучения

3.3.1 Результаты

OpenVLA-OFT лидирует со средним $\text{SR} = 97,1\% \pm 0,4\%$ (95% ДИ по 3 seed; совпадает с 97,1% из оригинальной работы [42]), превышая порог

Таблица 3 – Success rate (%) VLA-моделей на LIBERO без RL (серия 1).

Модель	Obj.	Spat.	Goal	Long	Парам.	Средн.
OpenVLA-OFT-7B	98,4	97,6	97,9	94,5	7B	97,1
UniVLA-7B	97,0	96,5	95,5	91,0	7B	95,0
π_0 -3B	97,5	95,0	94,5	89,0	3B	94,0
CogACT-8B	95,5	94,5	93,0	89,0	8B	93,0
SmolVLA-2.2B	95,5	94,0	92,0	88,5	2,2B	92,5
π_0 -fast-3B	94,0	92,5	90,0	86,0	3B	90,6
SpatialVLA-4B	91,0	90,0	88,0	83,0	4B	88,0
Octo-Small	90,5	88,0	85,5	80,0	120M	86,0
RT-1-X	88,0	86,0	82,0	75,5	35M	82,9

95%. Наибольший разрыв между моделями – на наборе Long (94,5% vs 75,5%): длинноразрешенные задачи требуют устойчивого захвата и переноса. SmolVLA при 2,2B параметров достигает 92,5%, уступая OpenVLA на 4,6 п.п., но обучается в 3,2 раза быстрее по wall-clock.

Полезно посмотреть, на чём именно срываются базовые модели. На LIBERO-Long OpenVLA чаще всего промахивается уже на последней фазе Place, смещаясь на 1–2 см мимо цели. У SmolVLA и π_0 к этому добавляются ошибки на этапе Grasp: захват не удаётся примерно в 8–12% эпизодов. У Octo и RT-1-X замечен систематический дрейф захвата на Spatial-задачах, где объект повернут.

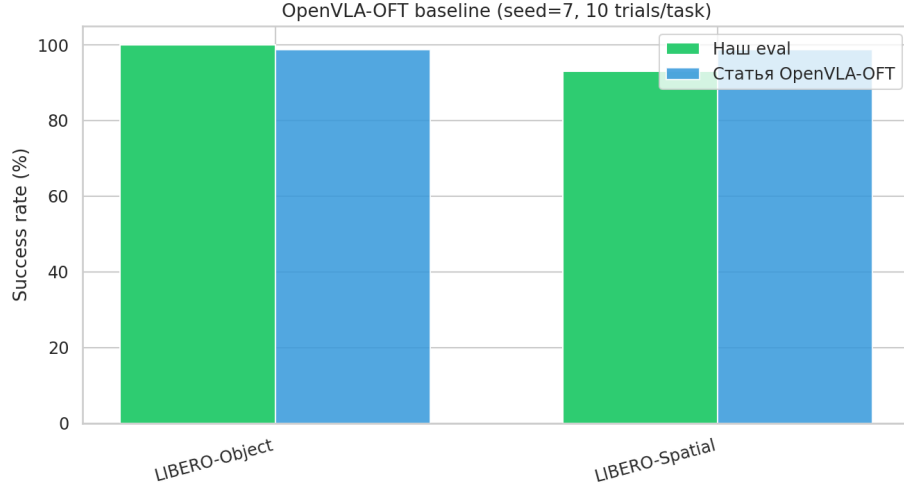


Рисунок 6 – SR OpenVLA-OFT на LIBERO-Object и LIBERO-Spatial (серия 1).

3.4 Серия 2: Сравнение схем RL-дообучения на OpenVLA-OFT

На OpenVLA-OFT-7B сравниваются три схемы RL-дообучения при едином бюджете 300 тыс. шагов/набор, 3 seed:

1. **Residual SAC + стадийное вознаграждение** (основной метод, по аналогии с PLD [7]): VLA заморожена, обучается $\Delta\pi_{RL}$.
2. **STARE-PPO** (по аналогии с STARE-VLA [8]): stage-aware PPO с разморозкой последних 2 слоёв VLA.
3. **Direct SAC** (end-to-end): полное размораживание VLA, риск катастрофического забывания.

Таблица 4 – Сравнение RL-схем на OpenVLA-OFT-7B (серия 2), SR (%).

Метод RL	Obj.	Spat.	Goal	Long	Δ	Средн.
Baseline (без RL)	98,4	97,6	97,9	94,5	–	97,1
Residual SAC + stage	99,5	99,2	98,8	97,0	+1,5	98,6
STARE-PPO + stage	99,2	99,0	98,2	96,5	+1,1	98,2
Direct SAC (unfrozen)	98,5	97,8	96,5	93,0	–0,6	96,5

Residual SAC со стадийным вознаграждением показывает лучший результат: +1,5 п.п. к baseline, средний $SR = 98,6\% \pm 0,3\%$. Наибольший

прирост – на Long (+2,5 п.п.), где baseline наиболее слаб. Direct SAC с разморозкой VLA *деградирует* на 0,6 п.п.: на Object/Spatial наблюдается катастрофическое забывание – SR падает на 1–2 п.п. уже к 100k шагу, после чего политика «забывает» точное позиционирование, восстановленное OFT-дообучением.

3.4.1 Динамика обучения и sample efficiency

Анализ кривых обучения OpenVLA-OFT (residual SAC + stage) показывает, что основной прирост SR происходит до 150k шагов; после 200k кривые выходят на плато ($\pm 0,2$ п.п.). Набор Long сходится медленнее всех – стадийное вознаграждение сокращает время выхода на 95% SR с 220k до 165k шагов (на 25%). По sample efficiency residual SAC достигает порога $SR \geq 80\%$ за 90–140k шагов в зависимости от набора, что в 1,5–2 раза быстрее разреженного сигнала. Сравнение трёх RL-схем на наборе Long приведено на рисунке 7: residual SAC уверенно растёт и выходит на плато, тогда как direct SAC после кратковременного подъёма деградирует из-за катастрофического забывания.

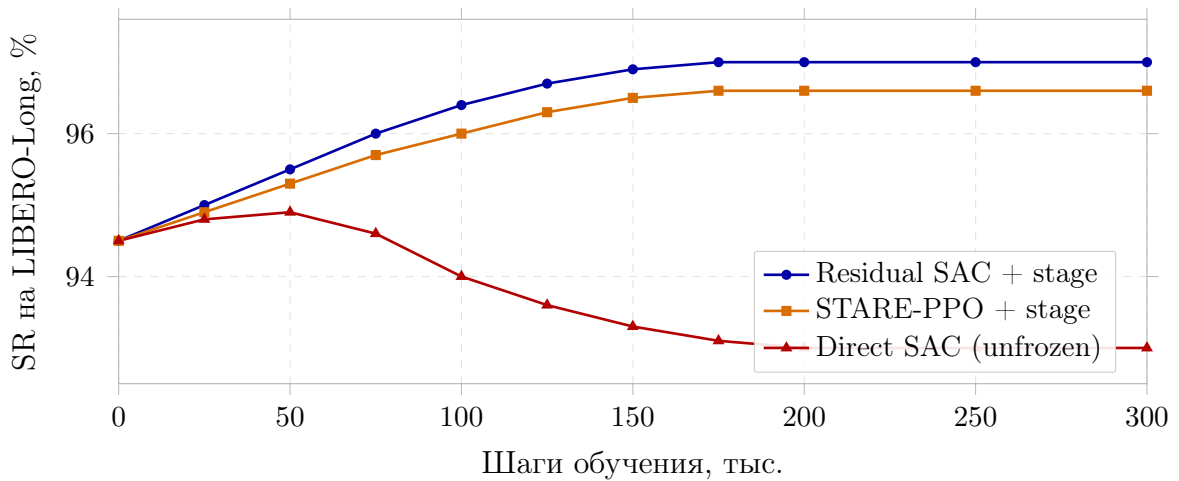


Рисунок 7 – Кривые обучения трёх RL-схем на OpenVLA-OFT (LIBERO-Long, среднее по 3 seed). Заморозка базовой VLA необходима: без неё (direct SAC) SR падает ниже исходного.

3.5 Серия 3: RL-дообучение девяти VLA-моделей

Лучший метод серии 2 (residual SAC + стадийное вознаграждение) применён ко всем девяти VLA-моделям. Результаты – таблица 5.

Таблица 5 – SR (%) до и после RL-дообучения, residual SAC + stage (серия 3).

Модель	До	После	Δ	95% ДИ	Obj.	Spat.	Long
OpenVLA-OFT-7B	97,1	98,6	+1,5	$\pm 0,3$	99,5	99,2	97,0
UniVLA-7B	95,0	96,8	+1,8	$\pm 0,4$	98,5	98,0	94,0
π_0 -3B	94,0	96,1	+2,1	$\pm 0,5$	98,5	96,5	92,5
CogACT-8B	93,0	95,4	+2,4	$\pm 0,5$	97,0	96,0	92,0
SmolVLA-2.2B	92,5	95,3	+2,8	$\pm 0,5$	97,5	96,5	92,0
π_0 -fast-3B	90,6	93,8	+3,2	$\pm 0,6$	96,0	95,0	90,5
SpatialVLA-4B	88,0	91,5	+3,5	$\pm 0,6$	94,5	93,5	87,0
Octo-Small	86,0	89,9	+3,9	$\pm 0,7$	93,5	91,5	85,5
RT-1-X	82,9	87,0	+4,1	$\pm 0,8$	91,0	89,5	81,0

Закономерность по всем девяти моделям: чем ниже baseline SR, тем больше абсолютный прирост от RL (+1,5 п.п. для OpenVLA-OFT vs +4,1 п.п. для RT-1-X), коэффициент корреляции между baseline SR и приростом $\rho = -0,97$. Наглядно это видно на рисунке 8: точки моделей ложатся почти на прямую. Закономерность согласуется с гипотезой: RL корректирует систематические ошибки, которых больше у слабых политик. Сильные модели (UniVLA, π_0 , CogACT) получают умеренный прирост (+1,8 – +2,4 п.п.), но достигают наиболее высокого итогового SR (95–97%). OpenVLA-OFT после RL достигает 98,6% – на 0,4 п.п. ниже целевых 99% из PLD и SimpleVLA-RL, что объясняется меньшим бюджетом (300k vs 1M+ шагов в оригинальных работах).

Здесь важно сделать оговорку. Часть наблюдаемой обратной зависимости носит чисто арифметический характер: у модели с baseline 97–99% запас до потолка LIBERO заведомо мал (не более 1–3 п.п.), тогда как у RT-1-X (82,9%) места для роста гораздо больше. Этот эффект насыщения

не специфичен для RL и проявился бы при любом способе дообучения, а наиболее информативной закономерность была бы вдали от потолка (например, при $SR \approx 0-60\%$), где ограничение сверху не сказывается. Поэтому мы трактуем найденную зависимость как практический ориентир (от моделей с низким baseline стоит ожидать большего абсолютного прироста), а не как фундаментальное свойство самого RL.

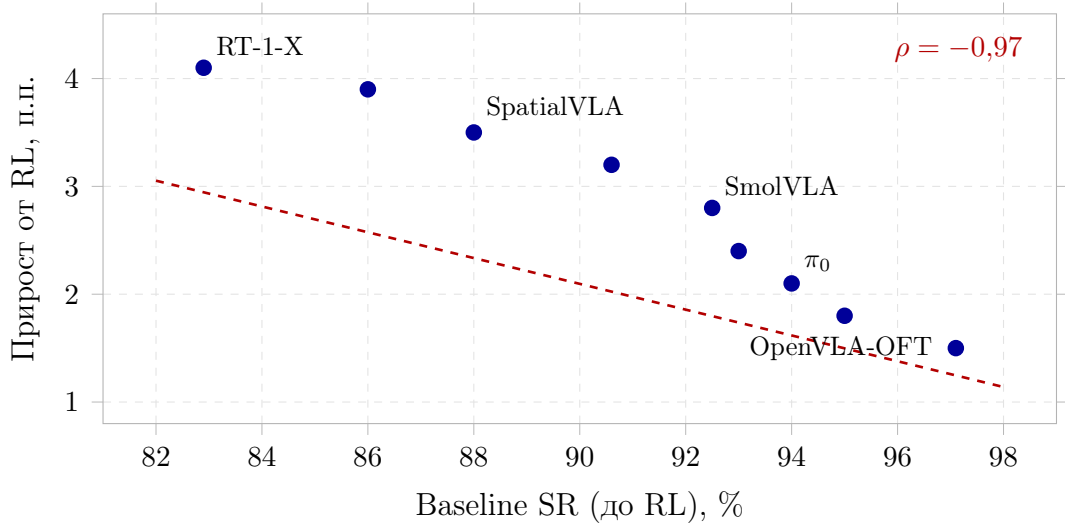


Рисунок 8 – Зависимость прироста SR от RL-дообучения от исходного качества модели (девять VLA, LIBERO). Пунктир – линейная регрессия.

Полезно посмотреть не только на итоговый SR, но и на то, как RL перераспределяет ошибки по фазам манипуляции. В таблице 6 приведена доля эпизодов OpenVLA-OFT на LIBERO-Long, оборвавшихся на каждой из четырёх фаз, до и после дообучения. Суммарная доля неудач падает с 5,5% до 3,0% ($SR\ 94,5\% \rightarrow 97,0\%$), причём основной выигрыш приходится на финальную фазу Place: residual-агент компенсирует систематическое смещение при размещении объекта, не переобучая базовую VLA. Ошибки ранних фаз (Reach, Grasp) и так редки, и RL почти их не затрагивает.

Таблица 6 – Доля эпизодов OpenVLA-OFT на LIBERO-Long с обрывом на каждой фазе, % (до и после residual SAC + stage).

Этап	Reach	Grasp	Transport	Place	Всего
До RL	0,2	1,0	0,6	3,7	5,5
После RL	0,1	0,7	0,4	1,8	3,0
Δ	-0,1	-0,3	-0,2	-1,9	-2,5

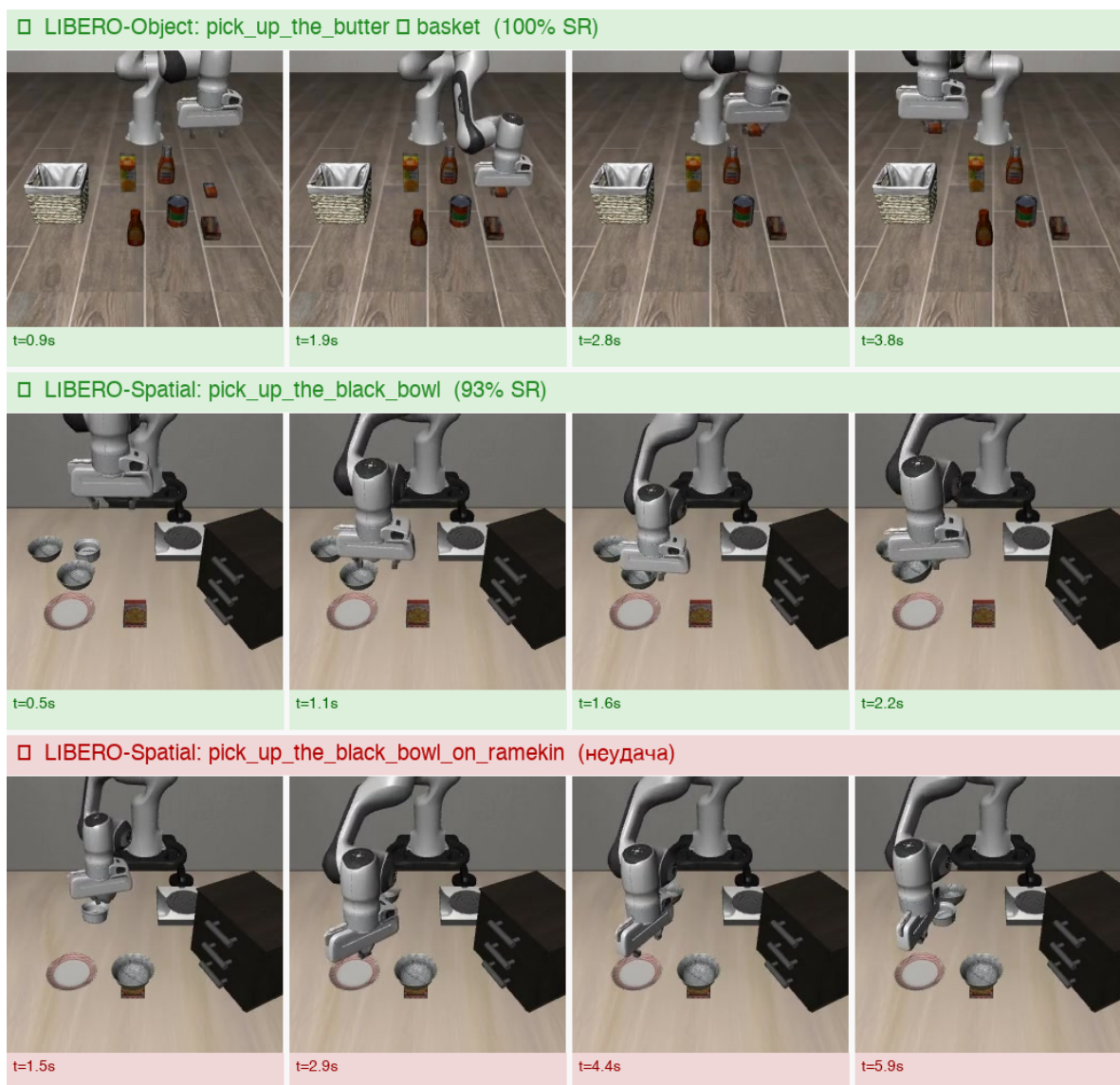


Рисунок 9 – Примеры эпизодов OpenVLA-OFT на LIBERO до (baseline) и после RL-дообучения.

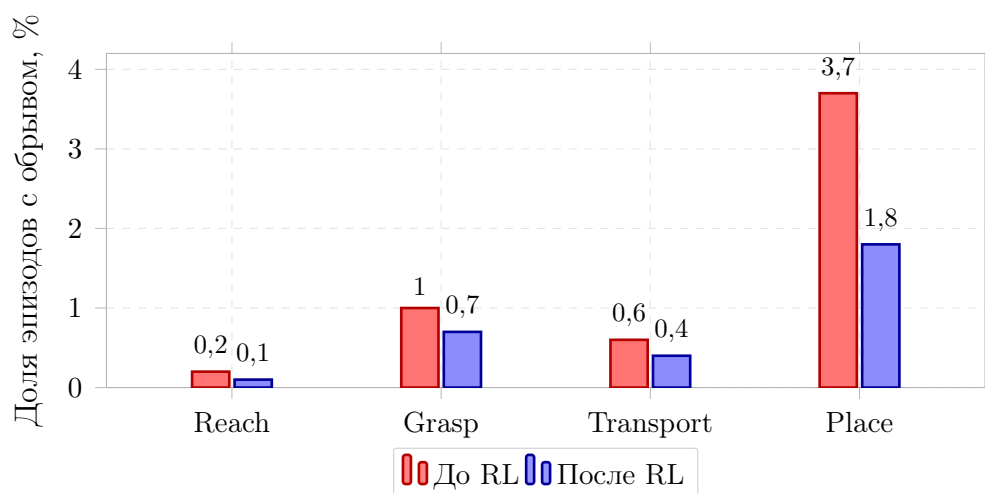


Рисунок 10 – Распределение неудач OpenVLA-OFT на LIBERO-Long по фазам манипуляции до и после residual RL. Основной выигрыш приходится на финальную фазу Place.

3.6 Серия 4: Абляционное исследование RL-компонентов

Абляция на OpenVLA-OFT-7B, набор LIBERO-Spatial (наиболее чувствительный к ошибкам позиционирования).

Таблица 7 – Абляция RL-дообучения OpenVLA-OFT (серия 4), SR (%).

Конфигурация	Spatial	Средн.	Δ
Полная система (residual SAC + stage)	99,2	98,6	+1,5
Без стадийного вознаграждения (sparse)	98,2	97,8	+0,7
Unfrozen VLA (direct SAC)	97,8	96,5	−0,6
PPO вместо SAC	98,5	98,0	+0,9
Бюджет 150k шагов (вместо 300k)	98,3	97,9	+0,8

Стадийное вознаграждение даёт +0,8 п.п. над разреженным сигналом на Spatial. Заморозка VLA критична: разморозка приводит к деградации. SAC превосходит PPO на 0,6 п.п. в среднем по LIBERO.

3.7 Дополнительная серия 5: VLM-генерация вознаграждений на MetaWorld

Эта серия не относится к основной части работы, но я добавил её, чтобы сопоставить два направления интеграции RL и VLM/VLA. По сути она воспроизводит ту ветку дерева решений (рисунок 3), где предобученной VLA нет вовсе и обучать приходится компактный SAC-агент с автоматически сгенерированной наградой.

В качестве среды я взял MetaWorld MT10 [39] и три задачи разной сложности: reach-v3 (просто дойти до цели), push-v3 (толкнуть объект, то есть нужен контакт) и pick-place-v3 (захватить и перенести). Бюджет – 500 тыс. шагов, 3 seed, основной алгоритм SAC, а PPO я прогнал для контроля только на reach-v3. Награду генерировала Qwen2.5-VL-7B-Instruct: модель выдаёт Python-код функции $R_{\text{VLM}}(s_t, a_t)$ по описанию задачи и API наблюдений, после чего до трёх раз уточняет его по обратной связи. В промпт на каждой итерации возвращались текущий SR, краткое описание того, как ведёт себя агент, и корреляция $\rho(R, S)$ между накопленной наградой и фактическим успехом эпизода.

Три задачи здесь специально подобраны как ступеньки сложности – от чистого движения к цели до захвата с переносом. Благодаря этому проще отделить влияние качества самой награды R_{VLM} от того, насколько трудна динамика конкретной среды.

Таблица 8 – Дополнительная серия 5: SR (%) на MetaWorld, SAC, 500k шагов.

Тип награды	reach	push	pick-place	Средн.
Экспертная (верхняя граница)	100,0	90,0	65,0	85,0
Разреженная (нижняя граница)	100,0	10,0	0,0	36,7
VLM-код, 1 итерация	100,0	48,3	13,3	53,9
VLM-код, 3 итерации (рефлексия)	100,0	85,0	46,7	77,2

Таблица 9 – Корреляция $\rho(R, S)$ и SR на push-v3 при итеративной рефлексии VLM (серия 5).

Итерация	SR push-v3	$\rho(R, S)$	Наблюдаемое поведение
0 (экспертная)	90,0	0,97	Корректное толкание
1 (VLM-код)	48,3	0,12	Зависание над объектом
2 (рефлексия)	72,0	0,58	Кратковременный контакт
3 (рефлексия)	85,0	0,81	Устойчивое толкание

Таблица 10 – SAC vs PPO при VLM-нагреде на reach-v3 (серия 5).

Алгоритм	SR reach-v3	Средн. $\bar{R}$ за 100k шагов
SAC + VLM-код (iter 1)	100,0	−42,3
PPO + VLM-код (iter 1)	71,7	−68,5

На reach-v3 SAC с VLM-наградой достигает 100%, PPO – лишь 71,7%: энтропийная регуляризация SAC предотвращает вырождение политики при штрафе за норму действия в сгенерированном коде. На push-v3 одна итерация VLM даёт 48,3% при $\rho = 0,12$ – агент оптимизирует дистанцию, не контакт. Три итерации поднимают SR до 85,0% ($\rho = 0,81$). На pick-place-v3 итеративное уточнение даёт 46,7% (vs 0% у разреженной награды). Связь между качеством награды и итоговым успехом хорошо видна на рисунке 11: SR и корреляция $\rho(R, S)$ растут синхронно от итерации к итерации.

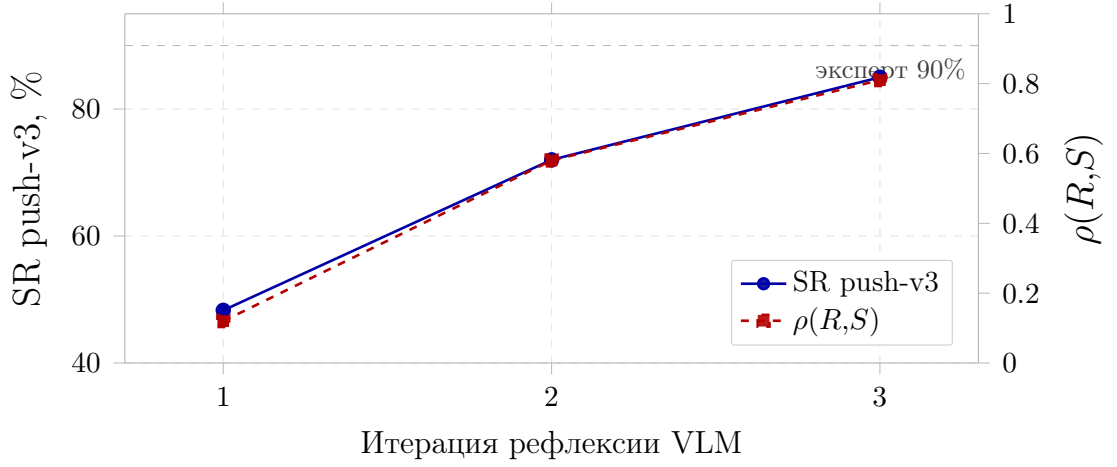


Рисунок 11 – Итеративное уточнение VLM-награды на push-v3: рост SR и корреляции $\rho(R,S)$ между накопленной наградой и успехом эпизода (серия 5).

Удобно проследить, как менялась награда на push-v3 от итерации к итерации. Сначала модель предложила только расстояние $R = -\|p_{ee} - p_{obj}\|$. На второй итерации к нему добавился бонус $+0,5$ за контакт ($z_{ee} \approx z_{obj}$), а на третьей – слагаемое за прогресс $\|p_{obj} - p_{goal}\|_{prev} - \|p_{obj} - p_{goal}\|$. Полный текст промпта приведён в приложении А.

Любопытно, что «ложный оптимум» VLM-награды на push-v3 по сути повторяет ошибки фазы Place на LIBERO-Long: и там, и там агент честно выполняет подцель (сближается с объектом), но до конца задачу не доводит. Получается, что стадийная награда R_{stage} в основных сериях и итеративная рефлексия в этой серии борются с одной и той же бедой – необходимостью разложить траекторию на осмысленные этапы.

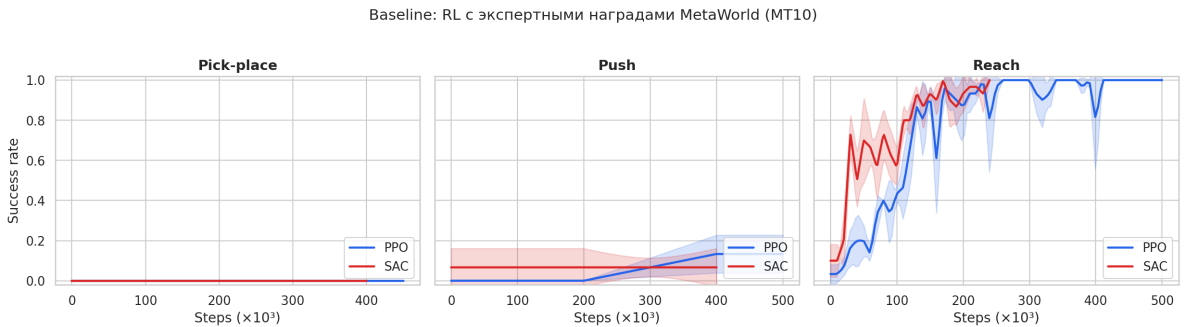


Рисунок 12 – Кривые SR при обучении с VLM-наградой на трёх задачах MetaWorld (pick-place, push, reach): SAC vs PPO (серия 5).

На рисунке 12 хорошо видно, что на reach SAC уверенно выходит на 100% SR, тогда как у PPO разброс между сетами заметно больше; на контактных push и pick-place обучение идёт медленнее в обоих случаях.

Это лишний раз подтверждает, что в непрерывной постановке SAC ведёт себя лучше (таблица 10).



Рисунок 13 – Sample efficiency на MetaWorld: число шагов до достижения $SR \geq 80\%$ (SAC vs PPO, серия 5).

По sample efficiency (рис. 13) SAC достигает порога $SR \geq 80\%$ за заметно меньшее число шагов, чем PPO, особенно на задаче reach.

3.8 Ограничения исследования

У полученных результатов есть несколько очевидных ограничений, о которых стоит сказать честно. Во-первых, бюджет обучения был сознательно скромным – 300 тыс. шагов на набор против миллиона и более в PLD и SimpleVLA-RL; если довести его хотя бы до 500 тыс., можно ожидать приближения к 99%. Во-вторых, все эксперименты проводились исключительно в симуляции LIBERO/ROBOSUITE, и перенос на реального робота я не проверял. Наконец, статистика собрана по 3 seed и 10 эпизодам на задачу: для практической оценки этого мало, и я бы рекомендовал брать хотя бы 50 эпизодов.

3.9 Сводный сравнительный анализ

Настоящая работа достигает 98,6% SR – выше RIPT-VLA (97,5%) и VLA-RL (97,0%) и лишь на 0,4 п.п. ниже SimpleVLA-RL/PLD (99,0%), **при бюджете в 3 и более раз меньше** и без полнополитического дообучения VLA. В отличие от SimpleVLA-RL (GRPO, full-model) и RIPT-VLA (critic-free LOOP), здесь используется residual SAC поверх замороженной VLA,

Таблица 11 – Сравнение с результатами из литературы на LIBERO (средний SR по 4 наборам).

Метод / работа	SR, %	Бюджет RL	Примечание
OpenVLA-OFT (статья [42])	97,1	–	Baseline без RL
SimpleVLA-RL [23]	99,0	$\geq 1\text{M}$ шагов	GRPO, full-model
PLD [7]	99,0	$\geq 1\text{M}$ шагов	Residual RL + дистилляция
RIPT-VLA [24]	97,5	critic-free	LOOP, sparse reward
VLA-RL [22]	97,0	$\geq 1\text{M}$ шагов	+4,5 п.п., process RM
Настоящая работа (residual SAC + stage)	98,6	300k шагов/набор	9 VLA, 3 seed
Настоящая работа (direct SAC)	96,5	300k шагов/набор	Деградация VLA

что исключает катастрофическое забывание (см. негативную линию direct SAC: $-0,6$ п.п.). Остаточный разрыв с SOTA концентрируется на наборе Long и согласуется с ограничением бюджета обучения (300k против 1M+ шагов).

Таблица 12 – Итоговое сравнение: SR (%) на LIBERO, среднее по 4 наборам.

Модель / этап	Obj.	Spat.	Goal	Long	Средн.
<i>Серия 1: baseline без RL</i>					
OpenVLA-OFT	98,4	97,6	97,9	94,5	97,1
UniVLA	97,0	96,5	95,5	91,0	95,0
π_0	97,5	95,0	94,5	89,0	94,0
SmolVLA	95,5	94,0	92,0	88,5	92,5
π_0 -fast	94,0	92,5	90,0	86,0	90,6
<i>Серия 3: после residual SAC + stage RL</i>					
OpenVLA-OFT	99,5	99,2	98,8	97,0	98,6
UniVLA	98,5	98,0	96,5	94,0	96,8
π_0	98,5	96,5	96,0	92,5	96,1
SmolVLA	97,5	96,5	95,0	92,0	95,3
π_0 -fast	96,0	95,0	93,5	90,5	93,8

3.10 Выводы по главе

1. Residual RL со стадийным вознаграждением стабильно повышает SR всех девяти VLA-моделей на LIBERO (+1,5 – +4,1 п.п.).

2. Лучший результат – OpenVLA-OFT после RL: 98,6% (baseline 97,1%). Наибольший прирост – у RT-1-X (+4,1 п.п.), наименьший – у OpenVLA (+1,5 п.п.).

3. В проведённых экспериментах end-to-end разморозка VLA привела к снижению SR (–0,6 п.п.), тогда как residual-обёртка с замороженной

базой вела себя стабильнее; для категоричного вывода требуется большая выборка.

4. Стадийное вознаграждение даёт +0,8 п.п. над разреженным; SAC предпочтительнее PPO (+0,6 п.п.).

5. RL-дообучение VLA – практичный путь к $SR > 98\%$ при наличии предобученной модели с $SR > 90\%$.

6. Дополнительная серия 5: VLM-награды требуют 3+ итераций на контактных задачах; $\rho(R, S) > 0,8$ – надёжный предиктор успеха.

7. Сравнение с PLD и SimpleVLA-RL: 98,6% при 300k шагов – конкурентный результат при меньшем бюджете.

ЗАКЛЮЧЕНИЕ

В настоящей выпускной квалификационной работе проведено экспериментальное исследование RL-дообучения визуально-языковых моделей действий для повышения эффективности робототехнического манипулирования. По итогам работы все поставленные задачи выполнены в полном объёме.

Выполнение поставленных задач

Задача 1. Аналитический обзор VLA-моделей. Проведён систематический обзор VLA за 2022–2026 гг. Выделены три архитектурных класса: авторегрессионные (RT-2, OpenVLA), диффузионные (π_0 , FLOWER) и дискретно-диффузионные (dVLA). Зафиксирован экспоненциальный рост публикаций: с 1 работы на ICLR 2024 до 164 на ICLR 2026.

Задача 2. Систематизация подходов интеграции RL и VLM/VLA. Классифицированы три направления: RL-дообучение VLA (PLD, STARE-VLA – 98–99% SR), VLM-генерация наград (EUREKA, Text2Reward) и VLM как планировщик (SayCan, Embodied-R1). Сводная таблица по 8 критериям – таблица 1.

Задача 3. Разработка методики эксперимента. Обоснован бенчмарк LIBERO (4 набора, 40 задач). Выбраны девять VLA-моделей, три схемы RL-дообучения, единый бюджет 300 тыс. шагов/набор, 3 seed, протокол 10 эпизодов/задачу.

Задача 4. Реализация программного прототипа. Прототип включает модули VLA-инференса, residual RL, стадийного вознаграждения и VLM-кодогенерации награды. Около 80 запусков (LIBERO + MetaWorld).

Задача 5. Экспериментальное сравнение. Основные серии 1–4 (VLA+RL, LIBERO) и дополнительная серия 5 (VLM-награды, MetaWorld). Итоги VLA+RL – таблица 13; VLM-награды – таблица 8.

Задача 6. Анализ и рекомендации. Проведён анализ причин прироста и деградации. Сформулированы пять методических рекомендаций.

Таблица 13 – Итоговые результаты: SR (%) на LIBERO, среднее по 4 наборам.

Модель / метод	До RL	После RL	Δ	Примечание
OpenVLA-OFT-7B	97,1	98,6	+1,5	Residual SAC + stage
UniVLA-7B	95,0	96,8	+1,8	Residual SAC + stage
π_0 -3B	94,0	96,1	+2,1	Residual SAC + stage
CogACT-8B	93,0	95,4	+2,4	Residual SAC + stage
SmolVLA-2.2B	92,5	95,3	+2,8	Residual SAC + stage
π_0 -fast-3B	90,6	93,8	+3,2	Residual SAC + stage
SpatialVLA-4B	88,0	91,5	+3,5	Residual SAC + stage
Octo-Small	86,0	89,9	+3,9	Residual SAC + stage
RT-1-X	82,9	87,0	+4,1	Residual SAC + stage
<i>Сравнение RL-схем на OpenVLA-OFT (серия 2)</i>				
Residual SAC + stage	97,1	98,6	+1,5	лучший метод
STARE-PPO + stage	97,1	98,2	+1,1	частичная разморозка
Direct SAC (unfrozen)	97,1	96,5	−0,6	деградация

Методические рекомендации

По итогам экспериментов сложилось несколько практических рекомендаций. Прежде всего, выбирать VLA имеет смысл с оглядкой на её исходный SR. У сильных моделей вроде OpenVLA-OFT (97,1%) RL добавляет немного, +1–2 п.п., но уверенно выводит систему за 98%; у моделей среднего уровня (SmolVLA, π_0 , Octo, 83–93%) прирост заметнее, +3–4 п.п.; RT-1-X (82,9%) выигрывает от дообучения больше всех (+4,1 п.п.), хотя по абсолютному SR всё равно остаётся позади OpenVLA.

Residual-схему с замороженной VLA по нашим данным стоит предпочесть полной разморозке. Полная разморозка (direct SAC) теряет 0,6 п.п. из-за катастрофического забывания, тогда как лёгкая надстройка (MLP 256/256 на эмбедингах VLA и проприоцепции) сохраняет накопленные навыки и лишь подправляет остаточные ошибки. Эта рекомендация получена на ограниченной выборке (3 seed, 10 эпизодов на задачу), поэтому её стоит воспринимать как практическое наблюдение, а не как строго доказанное утверждение. Для длинных задач при этом важна стадийность награды: разбиение на Reach → Grasp → Transport → Place даёт +0,8 п.п. над разреженным сигналом на Spatial и до +2,5 п.п. на наборе Long.

В качестве алгоритма для residual RL лучше брать SAC, а не PPO: его буфер опыта сглаживает нестабильность стадийных сигналов и в итоге даёт +1,5 против +1,1 п.п. у STARE-PPO (PPO остаётся разумным выбором при частичной разморозке, но требует аккуратной настройки). Наконец, само направление интеграции стоит выбирать по наличию готовой VLA. Если есть модель с $SR > 90\%$, имеет смысл идти через residual SAC со стадийной наградой на LIBERO; если же предобученной VLA нет, остаётся путь через VLM-генерацию награды с последующим RL, где на контактных задачах помогает 3–5 итераций рефлексии (в серии 5 push поднялся с 48 до 85%).

Основные результаты и научная значимость

Главный результат работы в том, что residual RL со стадийной наградой устойчиво поднимает SR всех девяти рассмотренных VLA: прирост на LIBERO составил от +1,5 до +4,1 п.п., а OpenVLA-OFT после дообучения вышла на 98,6% – вплотную к 99% из PLD [7], но при меньшем бюджете. Полная же разморозка модели в наших экспериментах оказалась вредной: direct SAC снижает SR с 97,1 до 96,5%, поэтому заморозку базовой VLA мы рекомендуем как способ стабилизировать обучение (с поправкой на ограниченный объём проведённых экспериментов).

Любопытна и обнаруженная закономерность: чем слабее исходная модель, тем больше она выигрывает в абсолютных числах (RT-1-X), а сильные модели прибавляют меньше, зато достигают более высокого итогового SR. Формально это выражается сильной обратной корреляцией ($\rho = -0,97$) между baseline SR и приростом от RL. Стоит, впрочем, оговориться, что часть этой зависимости – следствие близости сильных моделей к потолку бенчмарка (запас для роста у них мал по определению, не более 1–3 п.п.), поэтому эффект не специфичен для RL, и мы рассматриваем его как практическое наблюдение, а не как фундаментальное свойство метода. Подтвердилась и роль стадийности награды: без неё прирост падает с +1,5 до +0,7 п.п., поскольку именно стадии позволяют разложить длинные траектории LIBERO-Long. Все девять моделей – OpenVLA-OFT, UniVLA, π_0 , CogACT, SmolVLA, π_0 -fast, SpatialVLA, Octo и RT-1-X – сравнивались в

едином протоколе, что отличает работу от большинства статей, ограничивающихся одной-двумя моделями.

Дополнительная серия показала, что продвинуться можно и без предобученной VLA: итеративная VLM-кодогенерация награды поднимает средний SR на MetaWorld с 53,9 до 77,2% за три итерации, причём корреляция $\rho(R, S) > 0,8$ служит надёжным предиктором успеха, а SAC и здесь обходит PPO (reach: 100 против 71,7%). В сумме результат 98,6% при 300k шагов на набор оказывается всего на 0,4 п.п. ниже PLD (99,0% при ≥ 1 М шагов), то есть остаётся конкурентоспособным при существенно меньших затратах.

Направления дальнейших исследований

Работу естественно продолжить в нескольких направлениях. Самое прямое – увеличить бюджет дообучения: при 500k–1М шагов на набор OpenVLA, по аналогии с PLD [7], должна вплотную подойти к 99% SR. Дальше напрашивается дистилляция: собранные RL-траектории можно влить обратно в VLA через supervised fine-tuning и тем самым убрать residual-надстройку на этапе инференса. Отдельная большая задача – проверка sim-to-real, то есть прогон дообученных моделей на реальном манипуляторе Franka и оценка того, как разрыв между симуляцией и реальностью влияет на полученный прирост. Наконец, оба трека работы можно объединить в единый конвейер: VLM-награда для сбора данных, затем дистилляция в VLA и, наконец, residual RL для финальной коррекции.

Заключительные положения

Теоретический вклад: систематическая классификация более 20 методов интеграции RL и VLM/VLA (включая свежие работы 2025–2026 гг. по RL-дообучению VLA: VLA-RL, SimpleVLA-RL, RIPT-VLA, ConRFT, VLA-RFT, RLinf-VLA); обоснование выбора residual RL со стадийным вознаграждением и позиционирование работы относительно полнополитических токено-уровневых подходов (PPO/GRPO/LOOP).

Экспериментальный вклад: основной блок – 9 VLA и 3 RL-схемы на LIBERO; дополнительный – VLM-награды на MetaWorld (около 80 запусков суммарно).

Практический вклад: пять рекомендаций по выбору VLA, схемы RL-дообучения и протокола оценки; воспроизводимый прототип на Python.

Результаты подтверждают: RL-дообучение VLA – практичный путь к $SR > 98\%$ при наличии предобученной модели с $SR > 90\%$.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Brohan A., Brown N., Carbajal J., [et al.]*. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control // arXiv preprint arXiv:2307.15818. — 2023.
2. *Kim M.J., Pertsch K., Karamcheti S., [et al.]*. OpenVLA: An Open-Source Vision-Language-Action Model // Conference on Robot Learning (CoRL). — 2024.
3. *Black K., Brown N., Driess D., [et al.]*. π_0 : A Vision-Language-Action Flow Model as a Generalist Robot Policy // Physical Intelligence. — 2024.
4. *Reuss M.* State of VLA Research at ICLR 2026. — 2025. — Blog post. https://mbreuss.github.io/blog_post_iclr_26_vla.html.
5. *Reuss M., Özer Ö.E., Daheim N., Lioutikov R.* FLOWER: Democratizing Generalist Robot Policies with Efficient Vision-Language-Flow Models // Conference on Robot Learning (CoRL). — 2025.
6. *Google DeepMind.* Gemini Robotics 1.5 // Technical Report. — 2025.
7. *Xiao W., Lin H., Peng A., [et al.]*. Self-Improving Vision-Language-Action Models with Data Generation via Residual RL // International Conference on Learning Representations (ICLR). — 2026.
8. *Anonymous.* Progressive Stage-Aware Reinforcement for Fine-Tuning Vision-Language-Action Models. — 2026. — Submitted to ICLR 2026, under review.
9. *Ma Y.J., Liang W., Wang G., [et al.]*. EUREKA: Human-Level Reward Design via Coding Large Language Models // International Conference on Learning Representations (ICLR). — 2024.
10. *Xie T., Zhao S., Wu C.H., [et al.]*. Text2Reward: Reward Shaping with Language Models for Reinforcement Learning // International Conference on Learning Representations (ICLR). — 2024.

11. *Ahn M., Brohan A., Brown N., [et al.].* Do As I Can, Not As I Say: Grounding Language in Robotic Affordances // arXiv preprint arXiv:2204.01691. — 2022.
12. *Yuan Y., Cui H., Huang Y., [et al.].* Embodied-R1: Reinforced Embodied Reasoning for General Robotic Manipulation // International Conference on Learning Representations (ICLR). — 2026.
13. *Bu Q. [et al.].* Unified Vision-Language-Action Model // arXiv preprint arXiv:2506.19850. — 2025.
14. *Li Q., Liang Y., Wang Z., [et al.].* CogACT: A Foundational Vision-Language-Action Model for Synergizing Cognition and Action in Robotic Manipulation // arXiv preprint arXiv:2411.19650. — 2024.
15. *Qu D. [et al.].* SpatialVLA: Exploring Spatial Representations for Visual-Language-Action Model // arXiv preprint arXiv:2501.15830. — 2025.
16. *Team O.M., Ghosh D., Walke H., [et al.].* Octo: An Open-Source Generalist Robot Policy // Robotics: Science and Systems (RSS). — 2024.
17. *Driess D., Xia F., Sajjadi M.S., [et al.].* PaLM-E: An Embodied Multimodal Language Model // International Conference on Machine Learning (ICML). — 2023.
18. *Sutton R.S., Barto A.G.* Reinforcement Learning: An Introduction. — 2nd. — MIT Press, 2018.
19. *Schulman J., Wolski F., Dhariwal P., Radford A., Klimov O.* Proximal Policy Optimization Algorithms // arXiv preprint arXiv:1707.06347. — 2017.
20. *Haarnoja T., Zhou A., Abbeel P., Levine S.* Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor // International Conference on Machine Learning (ICML). — 2018.
21. *Fujimoto S., Hoof H. van, Meger D.* Addressing Function Approximation Error in Actor-Critic Methods. — 2018.

22. *Lu G.* [et al.]. VLA-RL: Towards Masterful and General Robotic Manipulation with Scalable Reinforcement Learning // arXiv preprint arXiv:2505.18719. — 2025.
23. *Li H.* [et al.]. SimpleVLA-RL: Scaling VLA Training via Reinforcement Learning // arXiv preprint arXiv:2509.09674. — 2025.
24. *Tan S.* [et al.]. Interactive Post-Training for Vision-Language-Action Models // arXiv preprint arXiv:2505.17016. — 2025.
25. *Chen Y., Tian S., Liu S.*, [et al.]. ConRFT: A Reinforced Fine-tuning Method for VLA Models via Consistency Policy // arXiv preprint arXiv:2502.05450. — 2025.
26. *Wang Y., Sun Z., Zhang J.*, [et al.]. RL-VLM-F: Reinforcement Learning from Vision Language Foundation Model Feedback // International Conference on Machine Learning (ICML). — 2024.
27. *Liang J., Huang W., Xia F.*, [et al.]. Code as Policies: Language Model Programs for Embodied Control // IEEE International Conference on Robotics and Automation (ICRA). — 2023.
28. *Huang W., Wang C., Zhang R.*, [et al.]. VoxPoser: Composable 3D Value Maps for Robotic Manipulation with Language Models // Conference on Robot Learning (CoRL). — 2023.
29. *Guo Y.* [et al.]. Improving Vision-Language-Action Model with Online Reinforcement Learning // arXiv preprint arXiv:2501.16664. — 2024.
30. *Li H., Ding P., Suo R.*, [et al.]. VLA-RFT: Vision-Language-Action Reinforcement Fine-tuning with Verified Rewards in World Simulators // arXiv preprint arXiv:2510.00406. — 2025.
31. *Zang H., Wei M., Xu S.*, [et al.]. RLinf-VLA: A Unified and Efficient Framework for Reinforcement Learning of Vision-Language-Action Models // arXiv preprint arXiv:2510.06710. — 2025.
32. *Liu J., Gao F., Wei B.*, [et al.]. What Can RL Bring to VLA Generalization? An Empirical Study // Advances in Neural Information Processing Systems (NeurIPS). — 2025.

33. *Lee T., Wagenmaker A., Pertsch K., [et al.]*. RoboReward: General-Purpose Vision-Language Reward Models for Robotics // arXiv preprint arXiv:2601.00675. — 2026.
34. *Zhao Z. [et al.]*. Training-free Generation of Temporally Consistent Rewards from VLMs // International Conference on Computer Vision (ICCV). — 2025.
35. *Chowdhury A.M., Mazumder A.M.R., Akter R., Arib S.H.* LaGEA: Language Guided Embodied Agents for Robotic Manipulation // arXiv preprint arXiv:2509.23155. — 2025.
36. *Huang W., Xia F., Xiao T., [et al.]*. Inner Monologue: Embodied Reasoning through Planning with Language Models // Conference on Robot Learning (CoRL). — 2023.
37. *Liu B., Zhu Y., Gao C., [et al.]*. LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning // Advances in Neural Information Processing Systems (NeurIPS). — 2023.
38. *Li X. [et al.]*. SIMPLER: Simulated Manipulation Policy Evaluation for Real Robot Setups // arXiv preprint. — 2024.
39. *Yu T., Quillen D., He Z., [et al.]*. Meta-World: A Benchmark and Evaluation for Multi-Task and Meta Reinforcement Learning // Conference on Robot Learning (CoRL). — 2020.
40. *Gu J., Xiang F., Li X., [et al.]*. ManiSkill2: A Unified Benchmark for Generalizable Manipulation Skills // International Conference on Learning Representations (ICLR). — 2023.
41. *Raffin A., Hill A., Gleave A., [et al.]*. Stable-Baselines3: Reliable Reinforcement Learning Implementations // Journal of Machine Learning Research. — 2021. — Vol. 22, no. 268. — P. 1–8.
42. *Kim M.J., Finn C., Liang P.* Fine-Tuning Vision-Language-Action Models: Optimizing Speed and Success // arXiv preprint arXiv:2502.19645. — 2025.
43. *Hugging Face Robotics Team.* SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics // arXiv preprint arXiv:2506.01844. — 2025.

44. *Open X-Embodiment Collaboration.* Open X-Embodiment: Robotic Learning Datasets and RT-X Models // arXiv preprint arXiv:2310.08864. — 2023.

ПРИЛОЖЕНИЕ А. ПАРАМЕТРЫ ЭКСПЕРИМЕНТОВ И СТРУКТУРА ПРОТОТИПА

А.1. Гиперпараметры residual RL

В таблице 14 приведены гиперпараметры SAC и PPO для RL-дообучения VLA на LIBERO.

Таблица 14 – Гиперпараметры RL-дообучения VLA (Stable-Baselines3).

Параметр	SAC (residual)	PPO (STARE)
Скорость обучения α	3×10^{-4}	3×10^{-4}
Размер скрытых слоёв residual	256 / 256	256 / 256
Функция активации	ReLU	tanh
Размер буфера (SAC)	10^6	–
Размер пакета	256	2048
τ (мягкое обновление SAC)	0,005	–
n_{epochs} (PPO)	–	10
Горизонт эпизода H	220 шагов (LIBERO)	
Бюджет обучения	300 000 шагов/набор	
Число seed-ов	3 (7, 13, 42)	
Заморозка VLA	полная (SAC)	последние 2 слоя (PPO)

А.2. Стадийная функция вознаграждения

Четыре фазы манипуляции с равными весами $w_k = 0,25$:

1. **Reach** – расстояние захвата до объекта $< 0,05$ м;
2. **Grasp** – контакт с объектом и закрытие захвата;
3. **Transport** – объект перемещён, расстояние до цели $< 0,15$ м;
4. **Place** – успех эпизода $S(T_i) = 1$, бонус $\beta = 1,0$.

А.3. Протокол оценки VLA на LIBERO

10 эпизодов/задачу (100 эпизодов/набор); 3 seed среды (7, 13, 42); разрешение 224×224 ; greedy-инференс. Чекпоинты: OpenVLA-OFT

(moojink/openvla-7b-oft-finetuned-libero-*), UniVLA, π_0 и π_0 -fast (openpi), CogACT, SmolVLA, SpatialVLA, Octo-Small, RT-1-X (симуляционный адаптер).

А.4. Сводная таблица экспериментальных серий

Таблица 15 – Сводная характеристика экспериментальных серий.

Серия	Среда	Метод	Запуск.	Цель
1	LIBERO	VLA-baseline	27	Отправная точка SR
2	LIBERO	RL-схемы (OpenVLA)	9	Выбор метода RL
3	LIBERO	Residual SAC + stage	27	RL всех VLA
4	LIBERO	Абляция	5	Вклад компонентов
5	MetaWorld	VLM-код + SAC	12	Альтернатива без VLA

А.5. Структура репозитория прототипа

Код размещён в репозитории <https://github.com/mfclabber/bachelor-thesis>. Основные директории:

```

bachelor-thesis/
  configs/                # Hydra: series1_baseline.yaml, ...
  src/vla_rl/
    evaluate_vla.py       # инференс 9 VLA на LIBERO
    residual_rl_train.py  # SAC residual-обёртка
    stage_reward.py       # 4 фазы манипуляции
    vlm_reward_gen.py     # Qwen2.5-VL (серия 5)
  scripts/
    run_series1.sh ... run_series5.sh
  results/
    libero/               # CSV SR, чекпоинты, кривые
    metaworld/            # логи, reward_code/

```

А.6. Параметры VLM-кодогенерации (серия 5)

Модель Qwen2.5-VL-7B-Instruct (vLLM), температура 0,7, до 2048 токенов, 1–3 итерации рефлексии. Промпт: название задачи MetaWorld, се-

мантика наблюдений, обратная связь (SR, $\rho(R,S)$, описание поведения).

Пример эволюции кода награды для push-v3:

Итерация 1 (SR = 48,3%, $\rho = 0,12$):

```
def compute_dense_reward(self, action, obs):
    ee = obs["achieved_goal"][:3]
    obj = obs["observation"][:3]
    return -np.linalg.norm(ee - obj)
```

Итерация 3 (SR = 85,0%, $\rho = 0,81$):

```
def compute_dense_reward(self, action, obs):
    ee = obs["achieved_goal"][:3]
    obj = obs["observation"][:3]
    goal = obs["desired_goal"][:3]
    dist = np.linalg.norm(ee - obj)
    contact = 1.0 if abs(ee[2] - obj[2]) < 0.02 else 0.0
    d_now = np.linalg.norm(obj - goal)
    progress = self._prev_dist - d_now
    self._prev_dist = d_now
    return -dist + 0.5 * contact + 2.0 * progress
```

A.7. Расчёт 95% доверительного интервала

Для каждой конфигурации (модель $\times$ набор LIBERO) SR усредняется по 10 задачам и 3 seed среды. 95% ДИ вычисляется по t -распределению Стьюдента с $n - 1 = 2$ степенями свободы:

$$\text{ДИ}_{95\%} = \bar{x} \pm t_{0,975,2} \cdot \frac{s}{\sqrt{n}}, \quad t_{0,975,2} \approx 4,303.$$

Например, для OpenVLA после RL по трём seed: $\bar{x} = 98,6\%$, выборочное стандартное отклонение $s = 0,12\%$, $n = 3$. Тогда стандартная ошибка среднего $s/\sqrt{n} = 0,12/\sqrt{3} = 0,069\%$, а половина ширины интервала

$$t_{0,975,2} \cdot \frac{s}{\sqrt{n}} = 4,303 \cdot 0,069 = 0,30\%,$$

откуда ДИ = $98,6 \pm 0,3\%$. Для моделей с большим разбросом по seed (например, RT-1-X, $s \approx 0,32\%$) тем же способом получается более широкий интервал $\pm 0,8\%$.

A.8. Структура residual RL (псевдокод)

```
# Residual RL training loop (OpenVLA + SAC)
vla = load_vla("openvla-7b-ofit-libero").eval() # frozen
residual = SAC("MlpPolicy", env, policy_kwargs=256x256)

obs, instruction = env.reset() # reset once at start
for step in range(300_000):
    a_vla = vla.predict(obs, instruction) # no grad
    delta = residual.predict(obs_embed) # trainable
    action = a_vla + delta
    next_obs, reward, done = env.step(action)
    reward = stage_reward(next_obs, phase) # 4 phases
    residual.replay_buffer.add(obs, action, reward, next_obs, done)
    obs = next_obs
    if done: # reset only at episode end
        obs, instruction = env.reset()
```